Playwright Automation Test Scripts

Total Files: 66

Index

35. [Test] hardvssoftassertion.spec.ts
36. [Test] hiddendropdown.spec.ts
37. [Test] hooks.spec.ts
38. [Test] hooks2.spec.ts
39. [Test] html-runner.spec.ts
40. [Test] infinitescroll.spec.ts
41. [Test] infinitescrollfindbook.spec.ts
42. [Test] keyboardaction.spec.ts
43. [Test] locators.spec.ts
44. [Test] mouseaction.spec.ts
45. [Test] mytest.spec.ts
46. [Test] mytest2.spec.ts
47. [Test] paginationtable.spec.ts
48. [Test] paralleltest.spec.ts
49. [Test] parameterization.spec.ts
50. [Test] parametertest.spec.ts
51. [Test] paratestCSV.spec.ts
52. [Test] paratestJSON.spec.ts
53. [Test] paratestXLSX.spec.ts
54. [Test] pomtest.spec.ts
55. [Test] popups.spec.ts
56. [Test] reporter.spec.ts
57. [Test] screenshot.spec.ts
58. [Test] shadowdom.spec.ts
59. [Test] sorted.spec.ts
60. [Test] statictable.spec.ts
61. [Test] tabs.spec.ts
62. [Test] tagging.spec.ts
63. [Test] tracing.spec.ts
64. [Test] uploadfile.spec.ts
65. [Test] xpath.spec.ts
66. [Test] xpathaxes.spec.ts

1. [Page] File: cartpage.ts

```
import { Page, Locator } from '@playwright/test';
export class CartPage {
  private page: Page;
  private productNamesInCart: Promise<Array<Locator>>;

  constructor(page: Page) {
    this.page = page;
    // CSS selector to select all product name cells in the cart table
    this.productNamesInCart = this.page.locator('#tbodyid tr td:nth-child(2)').all();
  }
  // Method to check if a specific product is present in the cart
  async checkProductInCart(productName: string): Promise<boolean> {
    const products = await this.productNamesInCart;
    for (const product of products) {
      const name:any = (await product.textContent())?.trim();
      console.log(name);
      if (name === productName) {
        return true;
      }
    }
    return false;
  }
}
```

2. [Page] File: homepage.ts

```
import { Page, Locator } from '@playwright/test';
export class HomePage {
  private readonly page: Page;
  private readonly productList:Promise<Array<Locator>>;
  //private readonly productListLocator: string;
  private readonly addToCartButton: Locator;
  private readonly cartLink: Locator;
  constructor(page: Page) {
    this.page = page;
    // CSS selector targeting all product links under the product cards
    this.productList = this.page.locator('div#tbodyid div.card h4.card-title a').all();
    // 'Add to cart' button (exact match using text)
    this.addToCartButton = this.page.locator('a:has-text("Add to cart")');
    // Cart link in the top menu
    this.cartLink = this.page.locator('#cartur');
  }
  // Method to add a specific product to cart
  async addProductToCart(productName: string): Promise<void> {
    const productElements = this.productList;
    for (const product of await productElements) {
      const name = await product.textContent();
      if (name?.trim() === productName) {
        await product.click();

        break;
      }
    }

// Handle alert/dialog after clicking "Add to cart"
this.page.once('dialog', async (dialog) => {
  if (dialog.message().includes('added')) {
    await dialog.accept();
  }
});
// Click the "Add to cart" button
await this.addToCartButton.click();
}
// Method to navigate to the cart
async gotoCart() {
  await this.cartLink.click();
}
}
```

3. [Page] File: loginpage.ts

```
import { Page, Locator } from "@playwright/test";
export class LoginPage {
  private readonly page: Page;
  private readonly loginLink: Locator;
  private readonly userNameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  constructor(page: Page) {
    this.page = page;
    this.loginLink = this.page.locator('#login2');
    this.userNameInput = this.page.locator('#loginusername');
    this.passwordInput = this.page.locator('#loginpassword');
    this.loginButton = this.page.locator("button[onclick='logIn()']");
  }
  async clickLoginLink(): Promise<void> {
    await this.loginLink.click();
  }
  async enterUserName(username: string): Promise<void> {
  //await this.userNameInput.waitFor({ state: 'visible' });  // □    wait first
  await this.userNameInput.fill(username);                  // □    then fill
}
 async enterPassword(password: string): Promise<void> {
  //await this.passwordInput.waitFor({ state: 'visible' });
  await this.passwordInput.fill(password);
}
  async clickOnLoginButton(): Promise<void> {
    await this.loginButton.click();
  }
  async performLogin(username: string, password: string)
   {
    await this.clickLoginLink();
    await this.enterUserName(username);
    await this.enterPassword(password);
    await this.clickOnLoginButton();
  }
}
```

4. [Test] File: accessibility.spec.ts

```
//Accessibility Testing is a type of testing used to check whether a software application or website can be used by people with disabilities.
//ext that would be hard to read for users with vision impairments due to poor color contrast with the background behind it
//UI controls and form elements without labels that a screen reader could identify
//Interactive elements with duplicate IDs which can confuse assistive technologies
//-->WCAG -web content accessibility guidelines
import{test,expect} from '@playwright/test';
import AxeBuilder from "@axe-core/playwright";
test("Accessibility test",async({page},testinfo)=>{
   //await page.goto("https://demowebshop.tricentis.com/");
   await page.goto("https://www.w3.org/"); //full accessible no violation

//1.scanning detect all types of WCAG violoations
/*const accessibilityScanResults=await new AxeBuilder({page}).analyze();
console.log("Number of WCAG violations:",accessibilityScanResults.violations.length);
expect(accessibilityScanResults.violations).toEqual([]);
expect(accessibilityScanResults.violations.length).toEqual(0);*/
//2.scanning for few wcag violation
/*const accessibilityScanResults= await new
AxeBuilder({page}).withTags(['wcag2a','wcag2aa','wcag21a','wcag21aa']).analyze();
await testinfo.attach("accessibility-scan-results",{
   body:JSON.stringify(accessibilityScanResults,null,2),
   contentType:"application/json"});
console.log("Number of violations:",accessibilityScanResults.violations.length);
expect(accessibilityScanResults.violations.length).toEqual(0);*/
//3.scanning for fe wcag violation with rules
const accessibilityScanResults= await new AxeBuilder({page}).disableRules(['duplicate-id']).analyze();
await testinfo.attach("accessibility-scan-results",{
   body:JSON.stringify(accessibilityScanResults,null,2),
   contentType:"application/json"});
console.log("Number of violations:",accessibilityScanResults.violations.length);
expect(accessibilityScanResults.violations.length).toEqual(0);
});
```

5. [Test] File: action.spec.ts

```
//Action
//action is used to perform operations on the web elements
//like click, type, select etc.
//input text action
import {test, expect,Locator} from "@playwright/test";
test('Text input action demo', async ({page})=>{
await page.goto("https://testautomationpractice.blogspot.com/");
const textbox:Locator=page.locator('#name');
await expect(textbox).toBeVisible();
await expect(textbox).toBeEnabled();
const maxLength:string | null=await textbox.getAttribute("maxlength"); //returns the max length of the textbox
//await textbox.fill("Playwright Automation");
//await expect(textbox).toHaveValue("Playwright Automation");
expect(maxLength).toBe("15");
await textbox.fill("Jhon Cannady");
console.log("text content of Firstname:",await textbox.textContent());
console.log("value of Firstname:",await textbox.inputValue());//inputValue() returns the value entered in the
textbox
const enteredValue:string=await textbox.inputValue();
expect(enteredValue).toBe("Jhon Cannady");
await page.waitForTimeout(3000);
});
//Radio button action
//only used to execute only this perticular test
test('Radio button action demo', async ({page})=>{
await page.goto("https://testautomationpractice.blogspot.com/");
const maleRadio:Locator=page.locator('#male');
await expect(maleRadio).toBeVisible();
await expect(maleRadio).toBeEnabled();
await maleRadio.check();
expect(await maleRadio.isChecked()).toBe(true);
expect(maleRadio).toBeChecked
await page.waitForTimeout(3000);
console.log("check for radion button selected",await maleRadio.isChecked());
});
//check box action
test.only('Check box action demo', async ({page})=>{
await page.goto("https://testautomationpractice.blogspot.com/");
//using getByLablel specific check box
const sundaycheckbox:Locator=page.getByLabel('Sunday');
await sundaycheckbox.check();
expect(await sundaycheckbox.isChecked()).toBe(true);
await expect(sundaycheckbox).toBeChecked();
console.log("Check box is selected:",await sundaycheckbox.isChecked());
//select all the chek box
```

```
const days:string[]=['Sunday','Monday','Tuesday','Wednesday','Thursday','Friday','Saturday'];
const checkboxes:Locator[]=days.map(index=>page.getByLabel(index));
expect(checkboxes.length).toBe(7);
//make all check box marked
for(const checkbox of checkboxes)
{
   await checkbox.check();
   await expect (checkbox).toBeChecked();
}
await page.waitForTimeout(2000);
//uncheck last 3 checkbox and assert
for(const checkbox of checkboxes.slice(-3))
{
   await checkbox.uncheck();
   await expect(checkbox).not.toBeChecked();
}
await page.waitForTimeout(5000);
//Toggle checkbox:if checked uncheck,if unchecked ,check.
for(const checkbox of checkboxes)
{
   if (await checkbox.isChecked())
   {
   //only if unchecked
   await checkbox.uncheck();
   await expect(checkbox).not.toBeChecked();

   }
   else{
   //only if checked
   await checkbox.check();
   await expect(checkbox).toBeChecked();
}}
await page.waitForTimeout(2000);
//random check box select
const indexes=[1,3,6];
for(const i of indexes)
{
 await checkboxes[i].check();
 await expect(checkboxes[i]).toBeChecked();
}
await page.waitForTimeout(2000);
//select check box based on the label
const weekname:string="Friday";
for(const label of days)
{
   if(label.toLowerCase()===weekname.toLowerCase())
   {
```

```
    const checkbox=page.getByLabel(label);
    checkbox.check();
    await expect(checkbox).toBeChecked();
    }
}
await page.waitForTimeout(3000);
});
```

6. [Test] File: allurereport.spec.ts

```
//Allure Report is a popular tool used to create beautiful and detailed test reports for automation frameworks like Playwright, Selenium, Cypress, etc.import { test, expect } from '@playwright/test';
//install package
//-> npm install -D allure-playwright
//then install
//npm uninstall -g allure-commandline
//then download allure zip file place in program files then open enviornment variable and add the path for the allure report
//to genarate report
//-->  allure generate ./allure-results -o ./allure-report
//to open report
//--> allure open ./allure-report
//to overwrite the report
//--> allure generate ./allure-results -o ./allure-report --clean
import { test, expect } from '@playwright/test';
test.beforeEach('Launchinf app', async ({ page }) => {

await page.goto('https://demowebshop.tricentis.com/');
});
test('logotest', async ({ page }) => {
  await expect(page.locator("img[alt='Tricentis Demo Web Shop']")).toBeVisible();
});
test('title test', async ({ page }) => {
  expect(await page.title()).toContain("Demo Web Shop1");
});
test('search test', async ({ page }) => {
  await page.locator('#small-searchterms').fill("laptop"); // fill teh text in search box
  await page.locator("input[value='Search']").click(); // click on the button
  await expect.soft(page.locator('h2 a').nth(0)).toContainText("laptop", { ignoreCase: true });
});
```

7. [Test] File: annotation.spec.ts

```
//Annotations
//Annotations in Playwright are special markers added to tests to control their execution or provide additional information about them.
//hey help testers skip tests, focus on specific tests, mark expected failures, or categorize tests during execution.
//only,skip,fail,fixme,slow
import { test, expect } from '@playwright/test';
//only
//test.only('test1', async ({page}) => {
test('test1', async ({page}) => {
   await page.goto('https://www.google.com/');
    await expect(page).toHaveTitle('Google');
    console.log('test 1 is running...');
});
test.skip('test2', async ({page}) => {
   await page.goto('https://www.google.com/');
    await expect(page).toHaveTitle('Google');
    console.log('test 2 is running...');
});
//skip the test based on condition
test.skip('test3', async ({page,browserName}) => {
    test.skip(browserName==='chromium','this test skipped if browser is chromium')
   await page.goto('https://www.google.com/');
    await expect(page).toHaveTitle('Google');
    console.log('test 3 is running...');
});
//fail
test.fail('test4', async ({page}) => {
   await page.goto('https://www.google.com/');
    await expect(page).toHaveTitle('Google');
    console.log('test 4 is running...');
});
//fixme
test.fixme('test5', async ({page}) => {
   await page.goto('https://www.google.com/');
    //No assertion
    console.log('test 5 is running...');
});
//slow
test('test6', async ({page}) => {
    test.slow(); //triple the default timeout(default:30sec,after this 90sec)
    await page.goto('https://www.google.com/');
    await expect(page).toHaveTitle('Google');
    console.log('test 6 is running...');
});
```

8. [Test] File: assertion.spec.ts

```
//Assertions
//Assertions are used to verify that the application is in the expected state. They are used to check if an element is visible, if it has the correct text, if it is enabled, etc.
//Playwright provides a built-in assertion library which is used to perform assertions in the tests. The expect function is used to perform assertions in the tests. It takes a locator as an argument and returns an assertion object which has various methods to perform assertions on the locator.
import { test, expect } from '@playwright/test';
test('Playwright Assertions Demo', async ({ page }) => {
  await page.goto('https://demowebshop.tricentis.com/');
  // 1. Auto-retrying assertion (automatically retries until it passes or times out)
  await expect(page).toHaveURL('https://demowebshop.tricentis.com/'); // waits for correct URL
  // Auto-retry: waits for the element to be visible and have the expected text
  await expect(page.locator('text=Welcome to our store')).toBeVisible();
  await expect(page.locator("div[class='product-grid home-page-product-grid'] strong")).toHaveText('Featured products'); // waits for the text to be present
  // 2. Non-retrying assertion (executes immediately, no retry)
  const title = await page.title();
  expect(title.includes('Demo Web Shop')).toBeTruthy(); // no auto-retry
  const welcometext = await page.locator('text=Welcome to our store').textContent();
  expect(welcometext).toContain('Welcome'); // non-retrying
  // 3. Negating matcher
  //await expect(page.locator('text=Welcome to our store')).not.toBeVisible(); // auto-retry
  //expect(welcometext).not.toContain('Welcome'); // no auto-retry
  await page.waitForTimeout(5000);
});
```

9. [Test] File: authenticatepopup.spec.ts

```
//Authenticated popup test
// it is used to handle the authenticated popup in the browser
// it is used to handle the authenticated popup in the browser by providing the username and password in the URL
import { test, expect, Locator, Page, chromium } from '@playwright/test';
test('Authenticated popup demo', async ({ browser }) => {
   //  EXTRA ADDED HERE (authentication)
   const context = await browser.newContext({
      httpCredentials: {
         username: 'admin',
         password: 'admin'
      }
   });
   const page = await context.newPage();
   // Approach 2: directly pass login along with url
   await page.goto('https://the-internet.herokuapp.com/basic_auth');
   await page.waitForLoadState(); // wait for page loaded completely
   await expect(page.locator('text=Congratulations')).toBeVisible();
   await page.waitForTimeout(5000);
});
```

10. [Test] File: autoscroll.spec.ts

```
//Automatic scrolling test
// it is used to handle the automatic scrolling in the browser when the element is not visible in the viewport
import { test, expect } from '@playwright/test';
test('Automatic scrolling demo', async ({ page }) => {
await page.goto('https://demowebshop.tricentis.com/');
//footer element locator=auto scroll to the footer element
const footer:string=await page.locator('.footer-disclaimer').innerText();
console.log("Footer text: "+footer);
await page.waitForTimeout(5000);
});
test('scrolling inside dropdown', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  await page.locator('#comboBox').click();
  const option =page.locator('#dropdown div:nth-child(100)'); // locator for the 10th option in the dropdown
  console.log("Option captured from dropdown:",await option.innerText());
  await option.click(); // click on the 10th option in the dropdown
  await page.waitForTimeout(3000);
});
test.only('scrolling inside the table', async ({ page }) => {
  await page.goto('https://datatables.net/examples/basic_init/scroll_xy.html');
  const name=await page.locator('tbody tr:nth-child(10) td:nth-child(2)').innerText(); // locator for the 50th row
and 1st column in the table
 console.log("Name captured from table:",name);
 const email=await page.locator('tbody tr:nth-child(10) td:nth-child(9)').innerText(); // locator for the 50th row
and 2nd column in the table
 console.log("Email captured from table:",email);
});
```

11. [Test] File: Autosuggestdropdown.spec.ts

```
//Bootstrapdropdown
//1)static drop down(select tag)
//A static dropdown is simply a dropdown built using the HTML <select> tag`, where the options are already present in the DOM (they don’t load dynamically from an API).
//2)Dynamic drop down/Auto suggest drop down
//A dynamic dropdown is a dropdown NOT built with the <select> tag.
//Its options are usually loaded at runtime (via JavaScript / API) and rendered using tags like
//3)hidden drop down
//A hidden dropdown is a dropdown where the options exist in the DOM but are not visible until some user action (hover, click, focus).
//Automation fails if you try to click options before they become visible.
//Autosuggest Drop down
import{test,expect,Locator} from "@playwright/test";
test("Auto suggest drop down",async({page})=>{
await page.goto("https://www.flipkart.com/");
await page.locator("input[name='q']").fill("smart");
await page.waitForTimeout(5000);
//get all suggeted options -->ctrl+shift+p on DOM-->emulated focused page
const options:Locator=page.locator("ul>li");
const count=await options.count();
console.log("Number of suggested options:",count);
await page.waitForTimeout(3000);
//return the text of perticular item
//console.log("5th option:",await options.nth(5).innerText());
//printing all the suggested options in the console
for(let i=0;i<count;i++)
{
   //console.log("5th option:",await options.nth(i).innerText());
   console.log(await options.nth(i).textContent());
   //console.log(await options.nth(i).allTextContents());
}
await page.waitForTimeout(2000);
//select/click on the smartphone option
for(let i=0;i<count;i++)
{
  const text=await options.nth(i).innerText();
  if(text==='smartphone')
  {
   await options.nth(i).click();
   await page.waitForTimeout(2000);
   break;

  }
}
});
```

12. [Test] File: autowaiting.spec.ts

```
//auto waiting test
//auto waiting is a feature of playwright which automatically waits for the element to be visible, enabled and stable before performing any action on it
//this test verifies that the application correctly waits for the element to be visible, enabled and stable before performing any action on it
import { test, expect } from '@playwright/test';
test('Auto Waiting Test', async ({ page }) => {
//default timeout for this test is 30 seconds, if the element is not visible, enabled and stable within 30 seconds, the test will fail
//test.slow(); //mark this test as slow to increase the timeout for this test to 90 seconds only for this test since we are waiting for the element to be visible, enabled and stable before performing any action on it
//set the timeout for this test to 60 seconds only for this test since we are waiting for the element to be visible, enabled and stable before performing any action on it
    test.setTimeout(60000); //set the timeout for this test to 60 seconds
    await page.goto('https://demowebshop.tricentis.com/'); //navigate to the application under test
//assertion-auto wait works
    await expect(page).toHaveURL('https://demowebshop.tricentis.com/'); //assert the URL of the page
    await expect(page.locator('text=Welcome to our store')).toBeVisible({timeout:5000}); //assert the visibility of the element
//Action-auto wait works
//force:true is used to perform the action on the element even if it is not visible, enabled and stable
    await page.locator('#small-searchterms').fill('laptop',{force:true});  //fill the search input field with the search term
    await page.locator('.button-1.search-box-button').click({force:true}); //click on the search button

});
```

13. [Test] File: bootstrapdatepicker.spec.ts

```
import { test, expect } from '@playwright/test';
test('Booking.com Date Picker Test - Check-in and Check-out', async ({ page }) => {
  await page.goto('https://www.booking.com/');
  // Open calendar
  await page.getByTestId('searchbox-dates-container').click();
  // ================= CHECK-IN DATE =================
  const checkinYear = "2026";
  const checkinMonth = "June";
  const checkinDay = "20";
  // Wait for calendar header to appear
  await page.locator("h3[aria-live='polite']").first().waitFor({ state: 'visible' });
  let attempts = 0;
  while (attempts < 24) {
    const headerText = await page
      .locator("h3[aria-live='polite']")
      .nth(0)
      .textContent();
    if (!headerText) throw new Error('Check-in month header not found');
    const [currentMonth, currentYear] = headerText.split(" ");
    if (currentMonth === checkinMonth && currentYear === checkinYear) {
      break;
    }
    await page.locator('button[aria-label="Next month"]').click();
    attempts++;
  }
  if (attempts === 24) {
    throw new Error('Check-in month/year not reached');
  }
  // Select check-in day
  const checkinDates = await page
    .locator('table.b8fcb0c66a tbody')
    .nth(0)
    .locator('td')
    .all();
  let checkinDateSelected = false;
  for (const date of checkinDates) {
    const dateText = await date.innerText();
    const classes = await date.getAttribute('class');
    if (
      dateText === checkinDay &&
      !classes?.includes('b8fcb0c66a--disabled')
    ) {
      await date.click();
      checkinDateSelected = true;
      break;
```

```
  }
 }
 expect(checkinDateSelected).toBeTruthy();
 // ================= CHECK-OUT DATE =================
 const checkoutYear = "2026";
 const checkoutMonth = "July";
 const checkoutDay = "25";
 await page.locator("h3[aria-live='polite']").nth(1).waitFor({ state: 'visible' });
 attempts = 0;
 while (attempts < 24) {
  const headerText = await page
   .locator("h3[aria-live='polite']")
   .nth(1)
   .textContent();
  if (!headerText) throw new Error('Check-out month header not found');
  const [currentMonth, currentYear] = headerText.split(" ");
  if (currentMonth === checkoutMonth && currentYear === checkoutYear) {
   break;
  }
  await page.locator('button[aria-label="Next month"]').click();
  attempts++;
 }
 if (attempts === 24) {
  throw new Error('Check-out month/year not reached');
 }
 // Select check-out day
 const checkoutDates = await page
  .locator('table.b8fcb0c66a tbody')
  .nth(1)
  .locator('td')
  .all();
 let checkoutDateSelected = false;
 for (const date of checkoutDates) {
  const dateText = await date.innerText();
  const classes = await date.getAttribute('class');
  if (
   dateText === checkoutDay &&
   !classes?.includes('b8fcb0c66a--disabled')
  ) {
   await date.click();
   checkoutDateSelected = true;
   break;
  }
 }
 expect(checkoutDateSelected).toBeTruthy();
 await page.waitForTimeout(5000); // optional visual check
});
```

14. [Test] File: browsercontext.spec.ts

```
//Broiwser context test
// it is used to create incognito browser context and perform actions in that context
// it is used to create multiple browser contexts in the same test and perform actions in those contexts
import { test, expect,Locator,Page,chromium } from '@playwright/test';
/*test('Browser context demo', async ({ browser }) => {
chromium.launch(); // launch the browser
 const context=await browser.newContext(); // create a new incognito browser context
 const page=await context.newPage();
 await page.goto('https://testautomationpractice.blogspot.com/');*/
//creating multiple pages in the same browser context
test('Browser context demo', async () => {
const browser=await chromium.launch();
const context=await browser.newContext();
const page1=await context.newPage();
const page2=await context.newPage();
console.log("Number of pages created:",context.pages().length);
await page1.goto('https://playwright.dev/');
await expect(page1).toHaveTitle(/Playwright/);
await page1.waitForTimeout(5000);
await page2.goto('https://www.selenium.dev/');
await expect(page2).toHaveTitle(/Selenium/);
await page2.waitForTimeout(5000);
});
```

15. [Test] File: browsersetting.spec.ts

```
//Browser settings test
//This test verifies that the application correctly handles browser settings such as viewport size, user agent, and geolocation.
//Browser settings are important for testing the responsiveness of the application, simulating different devices, and testing location-based features.
import { test, expect ,chromium} from '@playwright/test';
test('Browser Settings Test', async () => {
//browser-->context-->page
const browser=await chromium.launch({headless:false}); //launch the browser in headed mode to see the changes in the browser settings
//const browser=await chromium.launch({headless:true}); //launch the browser in headless mode to run the tests in the background without opening the browser window
const context=await browser.newContext(
{
   viewport:{width:1200,height:800 }, //set the viewport size to 1200x800 pixels to test the responsiveness of the application
   locale:'en-US', //set the locale to English (United States) to test the application in different languages and regions
   //proxy:{server:'http://myproxy.com:8080',username:'user1',password:'password1'}, //set the proxy server to test the application in different network conditions
//SSL-secure socket layer
//ssl is a security protocol that provides secure communication over the internet by encrypting the data transmitted between the client and the server.
//It is commonly used to protect sensitive information such as login credentials, credit card details, and personal data from being intercepted by malicious actors.
// In Playwright, you can configure SSL settings to test how your application handles secure connections and to ensure that it works correctly with SSL certificates.
ignoreHTTPSErrors:true, //ignore HTTPS errors to test the application with self-signed certificates or in development environments where SSL is not properly configured
}
);
const page=await context.newPage();
//await page.goto("https://www.google.com/"); //navigate to the application under test
await page.goto("https://expired.badssl.com/"); //navigate to the application under test
console.log("Page title: ",await page.title()); //print the page title to verify that the page is loaded successfully
await page.waitForTimeout(3000); //wait for some time to see the changes in the browser settings
});
```

16. [Test] File: codegen.spec.ts

```
//test generator(codegen)
//it generates test code by recording user interactions with the browser.
//to start codegen, run: npx playwright codegen <url>
//to open the browser in codegen mode, run: npx playwright codegen --browser=chromium https://
www.saucedemo.com/
//to create the file automatically- npx playwright codegen -o tests/codegen.spec.ts
import { test, expect } from '@playwright/test';
test('test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await expect(page.getByRole('link', { name: 'PRODUCT STORE' })).toBeVisible();
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginpassword').click();
  await page.locator('#loginpassword').fill('test@123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await page.getByText('New message × Contact Email: Contact Name: Message: Close Send message Sign
up').click();
  await expect(page.getByRole('link', { name: 'Log out' })).toBeVisible();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');
  await page.getByRole('link', { name: 'Log out' }).click();
});
```

17. [Test] File: codegenbrowser.spec.ts

```
//to run on different browsers
//npx playwright codegen -o tests/codegen.spec.ts --browser=chromium https://www.saucedemo.com/
//npx playwright codegen -o tests/codegen.spec.ts --browser=firefox https://www.saucedemo.com/
//npx playwright codegen -o tests/codegen.spec.ts --browser=webkit https://www.saucedemo.com/
//to run on different channels of browsers
//npx playwright codegen -o tests/codegen.spec.ts --browser=chromium --channel=chrome https://
www.saucedemo.com/
//npx playwright codegen -o tests/codegen.spec.ts --browser=chromium --channel=msedge https://
www.saucedemo.com/
//to run in headless mode
//npx playwright codegen -o tests/codegen.spec.ts --headless https://www.saucedemo.com/
//to run in headed mode
//npx playwright codegen -o tests/codegen.spec.ts --headless=false https://www.saucedemo.com/
import { test, expect } from '@playwright/test';
test('test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginusername').press('Tab');
  await page.locator('#loginpassword').fill('test@123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');
  await page.getByRole('link', { name: 'Log out' }).click();
  await expect(page.getByRole('link', { name: 'PRODUCT STORE' })).toBeVisible();
});
```

18. [Test] File: codegenmobile.spec.ts

```
//to run on different devices
//npx playwright codegen -o tests/codegen.spec.ts --device="iPhone 12" https://www.saucedemo.com/
import { test, expect, devices } from '@playwright/test';
test.use({
  ...devices['iPhone 12'],
});
test('test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginusername').press('Tab');
  await page.locator('#loginpassword').fill('test@123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');
  await page.getByRole('link', { name: 'Log out' }).click();
});
```

19. [Test] File: codegenview.spec.ts

```
import { test, expect } from '@playwright/test';
test.use({
  viewport: {
    height: 720,
    width: 1280
  }
});
test('test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginusername').press('Tab');
  await page.locator('#loginpassword').fill('test@123');
  await page.locator('#loginpassword').press('Enter');
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.getByRole('link').filter({ hasText: /^$/ }).nth(1)).toBeVisible();
  await page.getByRole('link', { name: 'Phones' }).click();
  await page.getByRole('link', { name: 'Log out' }).click();
});
```

20. [Test] File: comparingmethod.spec.ts

```
//static web table:
import{test,expect, Locator} from "@playwright/test";
test("comparing methods",async({page})=>{
await page.goto('https://demowebshop.tricentis.com/');
const products:Locator=page.locator('.product-title');
//1)innerText() extracts plain text.eliminates white space and line breaks
//textContent() extracts text including hidden elements.includes extra whitesspace,line breaks,etc
//console.log(await products.nth(1).innerText());  //14.1 inch laptop
//console.log(await products.nth(1).textContent());
/*const count=await products.count();
for(let i=0;i<count;i++)
{
   //const productname:string=await products.nth(i).innerText();//extracts plain text.eliminates white space and
line breaks.
   //console.log(productname);
   //const productname:string | null=await products.nth(i).textContent();//extracts text including hidden
elements.includes extra whitesspace,line breaks,etc
   //console.log(productname);
  const productname:string | null=await products.nth(i).textContent();
  console.log(productname?.trim()); //trim to remove the extras.
}*/
//2)allInnerText()
//allTextContent()
console.log("Second methos Results:");
//const productNames:string[]=await products.allInnerTexts();
//console.log("Prodct name captured by allInnerText():",productNames);
/*const productNames:string[]=await products.allTextContents();
console.log("Prodct name captured by allTextContent():",productNames);
const productNameTrimmed:string[]=productNames.map(text=>text.trim());
console.log("Product name after trim:",productNameTrimmed);*/
//3) all()
const productslocator:Locator[]=await products.all();
console.log(productslocator);
//console.log(await productslocator[3].innerText());
/*for(let productloc of productslocator)
{
   console.log(await productloc.innerText());
}*/
//for in loop
for(let i in productslocator)
{
   console.log(await productslocator[i].innerText());
}
});
```

21. [Test] File: cookies.spec.ts

```
//cookies test
//cookies are small pieces of data that are stored on the client side by the browser. They are used to store user preferences, session information, and other data that can be used to personalize the user experience.
//In Playwright, you can use the page.context().cookies() method to get the cookies for the current page, and the page.context().addCookies() method to add cookies to the browser context.
import { test, expect,chromium } from '@playwright/test';
test('Cookies Test', async () => {
const browser=await chromium.launch({headless:false}); //launch the browser in headed mode to see the cookies in the browser
const context=await browser.newContext();
const page=await context.newPage();
context.addCookies([
   {name:"cookie1",value:"value1",domain:".automationpractice.pl",path:"/"},
   {name:"cookie2",value:"value2",domain:".automationpractice.pl",path:"/"}]); //add cookies to the browser context
console.log("Cookies added to the browser context..."); //print the status message
await page.goto('https://www.automationpractice.pl/index.php'); //navigate to the application under test
//get the cookies for the current page
const cookiesadded=await context.cookies(); //get the cookies for the current page
const retrievedcookie1=cookiesadded.find((i)=>i.name==='cookie1'); //find the cookie with name 'cookie1' in the cookies array
const retrievedcookie2=cookiesadded.find((i)=>i.name==='cookie2'); //find the cookie with name 'cookie2' in the cookies array
console.log("Retrieved Cookie 1: ",retrievedcookie1); //print the retrieved cookie 1
console.log("Retrieved Cookie 2: ",retrievedcookie2); //print the retrieved cookie 2
expect(retrievedcookie1?.value).toBe("value1"); //assert the value of cookie 1
expect(retrievedcookie2?.value).toBe("value2"); //assert the value of cookie 2
expect(retrievedcookie1).toBeDefined(); //assert that cookie 1 is defined
expect(retrievedcookie2).toBeDefined(); //assert that cookie 2 is defined
//get all cookies for the current page and print their names and values
console.log("total number of cookies:",cookiesadded.length); //print the status message
expect(cookiesadded.length).toBeGreaterThan(0); //assert that at least 2 cookies are present in the browser context
console.log("Cookies retrieved successfully...."); //print the status message
for(const cookie of cookiesadded)
   {
   console.log(`${cookie.name}: ${cookie.value}`); //print the name and value of each cookie
   }
//cleaer the cookies after the test is completed
await context.clearCookies(); //clear the cookies from the browser context
console.log("Cookies cleared from the browser context..."); //print the status message
//verify that the cookies are cleared from the browser context
const cookiesafterclear=await context.cookies(); //get the cookies for the current page after clearing the cookies
console.log("Cookies after clearing: ",cookiesafterclear.length); //print the cookies after clearing
expect(cookiesafterclear.length).toBe(0); //assert that no cookies are present in the browser context after
```

```
clearing the cookies
console.log("Cookies cleared successfully...."); //print the status message
await page.waitForTimeout(3000); //wait for some time to see the cookies in the browser
});
```

22. [Test] File: csslocator.spec.ts

```
//CSS
//css is cascading style sheets
//css is used to style html elements
//css is used to make html elements look better
//css is used to change the layout of html elements
//css is used to make html elements responsive
//css is used to add animations to html elements
//css is used to add transitions to html elements
//css is used to add effects to html elements
//css is used to create themes for html elements
//css is used to create custom styles for html elements
//types of css locators
// 1) absolute locator
// 2) relative locator
// 3) attribute locator
/* tag with id  tag#id  or #id */
/* tag with class  tag.class  or .class */
/* tag with attribute  tag[attribute='value']  or [attribute='value'] */
/* tag with multiple classes  tag.class1.class2  or .class1.class2 */
/* tag with multiple attributes  tag[attribute1='value1'][attribute2='value2']  or [attribute1='value1']
[attribute2='value2'] */
/* tag with pseudo-class  tag:pseudo-class  or :pseudo*/
// # represents id
// . represents class
import { expect,test } from '@playwright/test';
test("verify css locators",async({page})=>{
   await page.goto("https://demowebshop.tricentis.com/");
   //tag with id
   await expect(page.locator("input#small-searchterms")).toBeVisible();
   await page.locator("input#small-searchterms").fill("T-Shirts");
//tag with class
await page.locator("input.search-box-text").fill("T-Shirts");
//tag with attribute
//tag[attribute='value']
await page.locator(".search-box-text[value='Search store']").fill("T-Shirts");
});
//Absolute locator starts from the root (html) and follows the full DOM path to the element, making it brittle
and not recommended for automation.
//What “absolute locator” means
//It tells the browser/test tool:
//“Go from html → body → first div → next div → then p … exactly in this order.”
```

23. [Test] File: Datepicker.spec.ts

```
//Date pcikers
import { test, expect, Locator,Page } from '@playwright/test';
async function selectDate(targetyear:string,targetmonth:string,targetdate:string,page:Page,isFuture:boolean)
{
 while(true)
{
  const selectedmonth=await page.locator('.ui-datepicker-month').textContent();
  const selectedyear=await page.locator('.ui-datepicker-year').textContent();

  if(selectedmonth?.trim()===targetmonth && selectedyear?.trim()===targetyear)
  {
   break;
  }
 if(isFuture)
 {
 await page.locator('.ui-datepicker-next').click();  //Future
 }
 else{
 await page.locator('.ui-datepicker-prev').click();   //Past
 }
 //await page.waitForTimeout(2000);
 }
 /*const allDates=await page.locator(".ui-datepicker-calender td").all();
 for(let dates of allDates)
 {
  const dateText= await dates.innerText();
  if(dateText===date)
  {
   await dates.click();
   break;
  }

 }*/

 const allDates=await page.locator(".ui-datepicker-calendar td").all();
 for(let dates of allDates)
 {
 const dateText= await dates.innerText();
  if(dateText===targetdate)
  {
   await dates.click();
   break;
  }

 }
```

```
}
test("Date picker",async({page})=>{
await page.goto("https://testautomationpractice.blogspot.com/");
const dateInput:Locator=page.locator("#datepicker");
await expect(dateInput).toBeVisible();
//xpect(dateInput).toBeVisible();
//1)using fill() method
/*datepicker.fill("07/02/2026");   //mm/dd/yy
await page.waitForTimeout(5000);*/
await dateInput.click(); //opens date picker
/*//past target date
const year='2023';
const month='May';
const date='22';*/
//Future target date
const year='2028';
const month='July';
const date='14';
await selectDate(year,month,date,page,true);
const expecteddate='07/14/2028';
await expect(dateInput).toHaveValue(expecteddate);
await page.waitForTimeout(2000);
});
```

24. [Test] File: debugging.spec.ts

```
//debugging
//debugging is the process of identifying and fixing issues in your code. Playwright provides several tools and techniques to help you debug your tests effectively.
import { test, expect } from '@playwright/test';
test('test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginusername').press('Tab');
  await page.locator('#loginpassword').fill('test@123');
  await page.locator('#loginpassword').press('Enter');
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.getByRole('link').filter({ hasText: /^$/ }).nth(1)).toBeVisible();
  await page.getByRole('link', { name: 'Phones' }).click();
  await page.getByRole('link', { name: 'Log out' }).click();
});
```

25. [Test] File: dialogue.spec.ts

```
//handling daialogue
//alert, confirm, prompt
//by default dialogue is auto accepted, but we can change it to auto dismiss
//we can also listen to dialogue event and get the message and type of dialogue
import { test, expect } from '@playwright/test';
test('Handling Dialogues Test', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  page.on('dialog',(dialog)=> {
    console.log("Dialogue type is:",dialog.type());//return type of dialogue
    expect(dialog.type()).toContain("alert");//assert dialogue type
    console.log("Dialogue message is:",dialog.message());//return message of dialogue
    expect(`${dialog.message()}`).toContain("I am an alert box!");//assert dialogue message
    dialog.accept(); //auto accept dialogue
  });
  await page.locator("#alertBtn").click(); //open alert dialogue
  await page.waitForTimeout(5000);
});
test('Confirmation Dialogue Test', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  //Register a dialog handler for confirm dialogue
  page.on('dialog',(dialog)=> {
    console.log("Dialogue type is:",dialog.type());//return type of dialogue
    expect(dialog.type()).toContain("confirm");//assert dialogue type
    console.log("Dialogue text is:",dialog.message());//return message of dialogue
    expect(dialog.message()).toContain("Press a button!");//assert dialogue message
    dialog.dismiss(); //close the dialogue by clicking cancel
  });
  await page.locator("#confirmBtn").click(); //open confirm dialogue
  const text:string=await page.locator("#demo").innerText(); //get the text of element after closing the dialogue
  console.log("Text after closing the dialogue is:",text);
  await expect(page.locator("#demo")).toHaveText("You pressed Cancel!"); //assert the text after closing the
dialogue
  await page.waitForTimeout(5000);
});
test.only('Prompt Dialogue Test', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  page.on('dialog',(dialog)=> {
    console.log("Dialogue type is:",dialog.type());//return type of dialogue
    expect(dialog.type()).toContain("prompt");//assert dialogue type
    console.log("Dialogue text is:",dialog.message());//return message of dialogue
    expect(dialog.message()).toContain("Please enter your name:");//assert dialogue message
    expect(dialog.defaultValue()).toContain("Harry Potter");
    dialog.accept('John'); //close the dialogue by providing a value
  });
  await page.locator("#promptBtn").click(); //open prompt dialogue
```

```
  const text:string=await page.locator("#demo").innerText(); //get the text of element after closing the dialogue
  console.log("Text after closing the dialogue is:",text);
  await expect(page.locator("#demo")).toHaveText("Hello John! How are you today?"); //assert the text after closing the dialogue
  await page.waitForTimeout(5000);
});
```

26. [Test] File: downloadfile.spec.ts

```
//download files
//used to download files from the application under test
//since file download dialog box is a system level dialog box which cannot be handled by playwright, we can use the page.waitForEvent('download') method to wait for the download event and then use the download.saveAs() method to save the downloaded file to a specific location
import { test, expect } from '@playwright/test';
import fs from 'fs';
test('Text File Download', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/p/download-files_25.html');
    await page.locator("#inputText").fill("Hello World"); //provide some text to be downloaded
    await page.locator("#generateTxt").click(); //click on the generate text file button
    //start waiting for the download event before clicking on the download link
    const [download]=await Promise.all(
    [
     page.waitForEvent('download'), //wait for the download event
     page.locator("#txtDownloadLink").click() //click on the download text file button
    ]);
//save the downloaded file to a specific location
const downloadpath='downloads/HelloWorld.txt'; //provide the path where the downloaded file will be saved
await download.saveAs(downloadpath); //save the downloaded file to the specified location
//check if file exists in the specified location
const fileexists=fs.existsSync(downloadpath);//check if the file exists in the specified location
expect(fileexists).toBeTruthy(); //assert that the file exists in the specified location
console.log("File downloaded successfully...."); //print the status message
//cleanup - delete the downloaded file after the test is completed
if(fileexists)
{
    fs.unlinkSync(downloadpath); //delete the downloaded file
    console.log("Downloaded file deleted successfully...."); //print the status message
}
 await page.waitForTimeout(5000);
});
//pdf file download
test.only('PDF File Download', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/p/download-files_25.html');
    await page.locator("#inputText").fill("Hello World"); //provide some text to be downloaded
    await page.locator("#generatePdf").click(); //click on the generate text file button

    //start waiting for the download event before clicking on the download link
    const [download]=await Promise.all(
    [
     page.waitForEvent('download'), //wait for the download event
     page.locator("#pdfDownloadLink").click() //click on the download text file button
    ]);
//save the downloaded file to a specific location
```

```
const downloadpath='downloads/HelloWorld.pdf'; //provide the path where the downloaded file will be saved
await download.saveAs(downloadpath); //save the downloaded file to the specified location
//check if file exists in the specified location
const fileexists=fs.existsSync(downloadpath);//check if the file exists in the specified location
expect(fileexists).toBeTruthy(); //assert that the file exists in the specified location
console.log("File downloaded successfully...."); //print the status message
//cleanup - delete the downloaded file after the test is completed
if(fileexists)
{
   fs.unlinkSync(downloadpath); //delete the downloaded file
   console.log("Downloaded file deleted successfully...."); //print the status message
}
 await page.waitForTimeout(2000);
});
```

27. [Test] File: dropdown.spec.ts

```
//Static Drop down list:
import{test,expect,Locator} from '@playwright/test';
test("Single select drop down",async({page})=>{
 await page.goto('https://testautomationpractice.blogspot.com/');
//select option from the drop down
//by using visible text
await page.locator('#country').selectOption('india');
await page.waitForTimeout(2000);
//by using value attribute
await page.locator('#country').selectOption({value:'uk'});
await page.waitForTimeout(2000);
//by using label
await page.locator('#country').selectOption({label:'India'});
await page.waitForTimeout(2000);
//by using index
await page.locator('#country').selectOption({index:3});
await page.waitForTimeout(3000);
//check number of option in the dropdown(count)
const dropdownoption:Locator=page.locator('#country>option');
await expect(dropdownoption).toHaveCount(10);
const count=await dropdownoption.count();
console.log("Number of countries:",count);
await page.waitForTimeout(2000)
//check an option present in the drop down:
const optiontext:string[]=(await dropdownoption.allTextContents()).map(text=>text.trim());
console.log(optiontext);
expect(optiontext).toContain('Japan');
//printing option from the drop down
for(const option of  optiontext)
{
   console.log(option);
}
await page.waitForTimeout(3000);
});
//Multi drop down
test("Multi select drop down",async({page})=>{
 await page.goto('https://testautomationpractice.blogspot.com/');
//select multiple option
//using visible text
await page.locator('#colors').selectOption(['Red','Blue','Green']);
await page.waitForTimeout(3000);
//using vale attribute
await page.locator('#colors').selectOption(['red','green','white']);
await page.waitForTimeout(3000);
//by using label
```

```
await page.locator('#colors').selectOption([
  { value: 'red'},
  { value: 'green'},
  { value: 'yellow'}
]);
await page.waitForTimeout(3000);
//using index
await page.locator('#colors').selectOption([{index:0},{index:1}]);
await page.waitForTimeout(3000);
//check number of option in the dropdwon(count)
const dropdownoption: Locator = page.locator('#colors > option');
await expect(dropdownoption).toHaveCount(7);
await page.waitForTimeout(3000);
//Check an  option present in the drop down:
const optiontext: string[] = (await dropdownoption.allTextContents())
  .map(text => text.trim());
console.log(optiontext);
expect(optiontext).toContain('Green'); //check if the array contains green
await page.waitForTimeout(3000);
//printing option from the multi drop down
for(const option of optiontext)
{
   console.log(option);
}
await page.waitForTimeout(3000);
});
```

28. [Test] File: duplicatedropdown.spec.ts

```
//Duplicate drop down
import{test,expect,Locator} from '@playwright/test';
test("Single select drop down",async({page})=>{
 await page.goto('https://testautomationpractice.blogspot.com/');
//Having duplicate
const duplicateDropdown:Locator=page.locator('#colors>option');
//not having duplicate
//cons duplicateDropdown:Locator=page.locator('#animals>option');//try animals
const Dropdown:string[]=(await (duplicateDropdown.allTextContents())).map(text=>text.trim());
//array and tupple can have duplicate value
//Set cannot have duplicate value.
const myset=new Set<string>(); //set
const duplicates:string[]=[]; //array
for(const text of Dropdown)
{
   if(myset.has(text))
   {
        duplicates.push(text);
   }
   else{
         myset.add(text);
   }
}
console.log("Duplicated options are:",duplicates)
if(duplicates.length>(0))
{
   console.log("Duplicate option found:",duplicates);
}
else{
   console.log("No duplicate found..");
}
expect(duplicates.length).toBe(0);//2
});
```

29. [Test] File: dynamicelement.spec.ts

```
//The button text changes dynamically between START and STOP
//XPath uses logical OR:
//button[text()='STOP' or text()='START']
//Playwright automatically:
//Waits for the button
//Re-evaluates the XPath every time inside the loop
//using XPath to handle dynamic elements in Playwright
/*import { test, expect, Locator } from '@playwright/test';
test('Handle Dynamic Elements using XPath', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  // Loop to click the button 5 times
  for (let i = 1; i <= 5; i++) {
    // XPath to handle dynamic START / STOP button text
    const button: Locator = page.locator(
      "//button[text()='STOP' or text()='START']"
    );
    // Click the button
    await button.click();
    // Wait for 2 seconds
    await page.waitForTimeout(2000);
  }
});*/
//using CSS to handle dynamic elements in Playwright
/*import { test, expect, Locator } from '@playwright/test';
test('Handle Dynamic Elements using CSS', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
    // Loop to click the button 5 times
    for (let i = 1; i <= 5; i++) {
      // CSS to handle dynamic START / STOP button text
      const button: Locator = page.locator(
        'button#start_stop'
      );
        // Click the button
        await button.click();
        // Wait for 2 seconds
        await page.waitForTimeout(2000);
    }
});*/
//using Playwright specific locators to handle dynamic elements
//getByRole with regex to match dynamic button text
import { test, expect } from '@playwright/test';
// Using Playwright specific locators
test('Handle Dynamic Elements using PW Locators', async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  // Loop to click the button 5 times
```

```
  for (let i = 1; i <= 5; i++) {
    // Locate button by role and dynamic name
    const button = page.getByRole('button', { name: /START|STOP/ });
    // Click the button
    await button.click();
    // Wait for 2 seconds
    await page.waitForTimeout(2000);
  }
});
```

30. [Test] File: dynamictable.spec.ts

```
//Dyanamic table
//means data keeps changing
import{test,expect,Locator} from "@playwright/test";
test("Dyanamic table",async({page})=>{
await page.goto("https://practice.expandtesting.com/dynamic-table");
const table:Locator=page.locator("table.table tbody");
await expect(table).toBeVisible();
//select all the rows,then find number of rows
const rows:Locator[]=await table.locator("tr").all();
console.log("Number of rows in table:",rows.length);
expect(rows).toHaveLength(4);
//step1:for chrome process ges value of CPU load
//Read each row 0 check chrome presence.
let cpuLoad='';
for(const row of rows)
{
  const processrow:string=await row.locator("td").nth(0).innerText();
  if(processrow==="Chrome")
  {
      //cpuLoad=await row.locator('td:had-text("%")').innerText();
      cpuLoad=await row.locator('td:has-text("%")').innerText();
      console.log("CPU load of Chrome:",cpuLoad);
      break;
  }
}
//step 2: compare it with value in the yellow label.
let yellowboxtext:string=await page.locator("#chrome-cpu").innerText();
console.log("Chrome CPU load from yellow box:",yellowboxtext);
if(yellowboxtext.includes(cpuLoad))
{
   console.log("CPU load of chrome is equal.");
}
else{
   console.log("CPU load of chrome is not equal.");
}
expect(yellowboxtext).toContain(cpuLoad);
await page.waitForTimeout(5000);
});
```

31. [Test] File: example.spec.ts

```
import { test, expect } from '@playwright/test';
test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  // Expect a title "to contain" a substring.
  await expect(page).toHaveTitle(/Playwright/);
});
test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  // Click the get started link.
  await page.getByRole('link', { name: 'Get started' }).click();
  // Expects page to have a heading with the name of Installation.
  await expect(page.getByRole('heading', { name: 'Installation' })).toBeVisible();
});
```

32. [Test] File: flakytest.spec.ts

```
//flaky test
//flaky test is a test that fails randomly due to various reasons such as network issues, timing issues, or other external factors.
//  It is important to identify and fix flaky tests to ensure the reliability of your test suite.
//ways to handle flaky tests in playwright
//global level
//1. using retry option in the playwright.config.ts file -->retry: 3, this will retry the test 3 times before marking it as failed
//for a specific test
//npx playwright test flakytest.spec.ts  --retries=3
import { test, expect } from '@playwright/test';
test('Flaky Test', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginpassword').fill('test@123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await page.waitForTimeout(8000); //introducing a delay to simulate a flaky test
  await expect(page.getByRole('link', { name: 'Log out' })).toBeVisible();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');
});
```

33. [Test] File: frame.spec.ts

```
//frame
//frame is a class that represents a frame in the game
//it has a constructor that takes in the frame number and the rolls in the frame
//it has a method that calculates the score of the frame based on the rolls and the next rolls in the game
import { test, expect,Locator } from '@playwright/test';
import { url } from 'node:inspector';
test('Frames demo', async ({ page }) => {
  await page.goto('https://ui.vision/demo/webtest/frames/');
// total frames in the page
const frames=page.frames();
console.log("Total frames in the page:",frames.length);
/*//using frame locator to access the frame and perform actions
  const frame = page.frame({ url: /frame_1/ });
if(frame){
    await frame.locator(("[name='mytext1']")).fill("Hello");
    //await frame.fill("[name='mytext1']","Hello World");
}
else{
    console.log("Frame is not visible");
}
await page.waitForTimeout(5000);*/
//using frame locator to access the frame and perform actions
const frame=page.frameLocator("[src='frame_1.html']").locator("[name='mytext1']").fill("Hello World");
await page.waitForTimeout(5000);
});
test.only('inner child frames demo', async ({ page }) => {
  await page.goto('https://ui.vision/demo/webtest/frames/');
  // Get frame_3
  const frame3 = page.frame({ url: /frame_3/ });
  if (frame3) {
    // Fill textbox in frame_3
    await frame3.locator("[name='mytext3']").fill("Hello World");
    // Wait until child frame loads
    await page.waitForTimeout(2000);
    const childFrames = frame3.childFrames();
    console.log("Child frames count:", childFrames.length);
    if (childFrames.length > 0) {
      const child = childFrames[0];
      // Use exact label text (Google form is case sensitive)
      const radio = child.getByLabel("I am a human");
      await radio.check();
      await expect(radio).toBeChecked();
    } else {
      console.log("Child frame not found");
    }
```

```
  } else {
    console.log("Frame is not visible");
  }
  await page.waitForTimeout(5000);
});
```

34. [Test] File: grouping.spec.ts

```
//grouping
// grouing is a way to organize your tests into logical groups.
//  It helps to run specific groups of tests based on the requirement.
//ways to group tests in playwright
//1.using fullyallparallel: false, //done by rohith in config file
/*import { test, expect } from '@playwright/test';
test('test1', async () => {
   console.log('This is test 1');
});
test('test2', async () => {
   console.log('This is test 2');
});
test('test3', async () => {
   console.log('This is test 3');
});
test('test4', async () => {
   console.log('This is test 4');
});*/
//2.describe block
//to run--> npx playwright test grouping.spec.ts --grep Group1
import { test, expect } from '@playwright/test';
test.describe('Group1',async() => {
test('test1', async () => {
   console.log('This is test 1');
});
test('test2', async () => {
   console.log('This is test 2');
});
});
test.describe('Group2',async() => {
test('test3', async () => {
   console.log('This is test 3');
});
test('test4', async () => {
   console.log('This is test 4');
});
});
```

35. [Test] File: hardvssoftassertion.spec.ts

```
//Hard vs soft assertion
//Hard assertion is an assertion that will stop the execution of the test if it fails.
// It is used to verify that the application is in the expected state before performing any action on it. It is used to check if an element is visible, if it has the correct text, if it is enabled, etc.
//Soft assertion is an assertion that will not stop the execution of the test if it fails.
// It is used to verify that the application is in the expected state after performing an action on it. It is used to check if an element is visible, if it has the correct text, if it is enabled, etc.
//Playwright provides a built-in assertion library which is used to perform assertions in the tests.
import { test, expect } from '@playwright/test';
test('Hard vs Soft Assertion', async ({ page }) => {
  await page.goto('https://demowebshop.tricentis.com/');
//1. Hard Assertion
/*await expect(page).toHaveTitle('Demo Web Shop2'); //hard assertion, if the title is not correct, the test will fail and stop execution
await expect(page).toHaveURL('https://demowebshop.tricentis.com/'); //hard assertion, if the URL is not correct, the test will fail and stop execution
const logo=await page.locator("img[alt='Tricentis Demo Web Shop']");//click on the logo to navigate to the home page
await expect(logo).toBeVisible(); *///hard assertion, if the logo is not visible, the test will fail and stop execution
//2. Soft Assertion
await expect.soft(page).toHaveTitle('Demo Web Shop2'); //soft assertion, if the title is not correct, the test will fail but continue execution
await expect.soft(page).toHaveURL('https://demowebshop.tricentis.com/'); //hard assertion, if the URL is not correct, the test will fail and stop execution
const logo=await page.locator("img[alt='Tricentis Demo Web Shop']");//click on the logo to navigate to the home page
await expect.soft(logo).toBeVisible();
await page.waitForTimeout(5000);
});
```

36. [Test] File: hiddendropdown.spec.ts

```
//Auto suggest drop down;
import { test, expect, Locator } from '@playwright/test';
test("Bootstrap hidden drop down",async({page})=>{
await page.goto("https://opensource-demo.orangehrmlive.com/web/index.php/auth/login");
 //Login page:
await page.locator('input[name="username"]').fill('Admin');
await page.locator('input[name="password"]').fill('admin123');
await page.locator('button[type="submit"]').click();

//click on the PIM:
await page.getByText('PIM').click();
// //click on job title:
await page.locator('formm i').nth(2).click();
await page.waitForTimeout(3000);
//capture all the option from dropdown:
const options:Locator=page.locator("div[role='listbox'] span");
const count:number=await options.count();
console.log("Number of options in dropdown:",count);
//print all the options
console.log("All the text content:",await options.allTextContents());
console.log("printing all the options....");
for(let i=0;i<count;i++)
{
   //console.log(await options.nth(i).innerText());
   console.log(await options.nth(i).textContent());
}
//selecct/click on option
for(let i=0;i<count;i++)
{
   const text=await options.nth(i).innerText();
   if(text==='Automation Tester')
   {
      await options.nth(i).click();
      break;
   }
}
await page.waitForTimeout(5000);
});
```

37. [Test] File: hooks.spec.ts

```
//hooks
//hooks are used to run some code before or after a test or a group of tests
//hooks are used to set up the test environment and clean up the test environment
//types
//beforeAll – runs once before all tests
//afterAll – runs once after all tests
//beforeEach – runs before every test
//afterEach – runs after every test
import { test, expect } from '@playwright/test';
test.beforeAll('Beforeall',async()=>{
   console.log('This is beofre all.....');
});
test.afterAll('Afterall',async()=>{
   console.log('This is After all.....');
});
test.beforeEach('Beforeeach',async()=>{
   console.log("This is before each.....")
})
test.afterEach('Aftereeach',async()=>{
   console.log("This is after each.....")
})
test('test1', async () => {
   console.log('This is test 1');
});
test('test2', async () => {
   console.log('This is test 2');
});
test('test3', async () => {
   console.log('This is test 3');
});
test('test4', async () => {
   console.log('This is test 4');
});
```

38. [Test] File: hooks2.spec.ts

```
//Hooks example
import{test,expect,Page} from "@playwright/test";
let page: Page;
test.beforeAll("Open app",async({browser})=>{
   page=await browser.newPage();
   await page.goto("https://www.demoblaze.com/index.html");
})
test.afterAll("Close app",async()=>{
   await page.close();

});
test.beforeEach("Login",async()=>{
   await page.locator('#login2').click();
   await page.locator('#loginusername').fill('pavanol');
   await page.locator('#loginpassword').fill('test@123');
   await page.locator("button[onclick='logIn()']").click();
   await page.waitForTimeout(2000);
});
test.afterEach("Logout",async()=>{
   await page.locator("#logout2").click();
   await page.waitForTimeout(2000);
});
test.describe('mygroup',async()=>{
 test('Find NoOf products', async () => {
   const products = page.locator('#tbodyid .hrefch');
   const count = await products.count();
   console.log("Number of products:", count);
   await expect(products).toHaveCount(9);
});
test('Add Product to cart', async () => {
   await page.locator("text='Samsung galaxy s6'").click();
   // Handle alert before the click
   page.once('dialog', async (dialog) => {
      expect(dialog.message()).toContain('Product added');
      await dialog.accept();
   });
   await page.locator('.btn.btn-success.btn-lg').click();
});
});
```

39. [Test] File: html-runner.spec.ts

```
import { test, expect } from '@playwright/test';
test('Run local hosted HTML in Playwright', async ({ page }) => {
  await page.goto('http://127.0.0.1:5500/tests/testhtml.html');
  // getByRole (heading)
  await expect(
    page.getByRole('heading', { name: 'Playwright Locator Practice App' })
  ).toBeVisible();
  // getByText
  await expect(
    page.getByText('Practice all Playwright locators on one page')
  ).toBeVisible();
  // getByAltText
  await expect(
    page.getByAltText('Playwright Company Logo')
  ).toBeVisible();
  // getByLabel
  await page.getByLabel('First name:').fill('John');
  await page.getByLabel('Last name:').fill('Wick');
  await page.getByLabel('Email:').fill('john@test.com');
  // getByPlaceholder
  await page.getByPlaceholder('Enter first name').fill('Johnny');
  // getByRole (button)
  await page.getByRole('button', { name: 'Create Account' }).click();
  // getByTitle
  await expect(
    page.getByTitle('Click here to get help')
  ).toBeVisible();
  // getByTestId
  await expect(
    page.getByTestId('dashboard-card')
  ).toBeVisible();
});
```

40. [Test] File: infinitescroll.spec.ts

```
//infinte scroll test
// it is used to handle the infinite scrolling in the browser when the element is not visible in the viewport and we need to scroll down to load more elements
import { test, expect } from '@playwright/test';
test('Infinite scroll demo', async ({ page }) => {
   test.slow(); // slow down the test execution to see the scrolling effect
await page.goto('https://www.booksbykilo.in/new-books?pricerange=201to500');
//window.scrollTo(0, document.body.scrollHeight); // scroll to the bottom of the page to load more elements
let previousheight = 0;
while(true)
    {
      await page.evaluate(() => {
          window.scrollBy(0, window.innerHeight); // scroll to the bottom of the page
       // wait for 2 seconds to load more elements

      });
      await page.waitForTimeout(2000);
      //capture the current height of the page after scrolling
      const currentheight =await page.evaluate(() => {
        return document.body.scrollHeight; // scroll to the bottom of the page to load more elements
      })
      console.log("Previous height: ",previousheight);
      console.log("Current height: ",currentheight);
      if(currentheight === previousheight)
      {
         break;
      }
      previousheight = currentheight;
   }
console.log("Reached the end of the page");

});
```

41. [Test] File: infinitescrollfindbook.spec.ts

```
import { test, expect } from '@playwright/test';
test('To find a book in infinite scroll demo', async ({ page }) => {
   test.slow(); // slow down the test execution to see the scrolling effect
await page.goto('https://www.booksbykilo.in/new-books?pricerange=201to500');
//window.scrollTo(0, document.body.scrollHeight); // scroll to the bottom of the page to load more elements
let previousheight = 0;
let bookFound = false;
while(true)
    {
const title = await page.locator('#productsDiv h3').allTextContents();
if(title.includes("The Ink Black Heart"))
{
   console.log("Book found");
   bookFound = true;
   expect(bookFound).toBeTruthy();
   break;
}
      await page.evaluate(() => {
         window.scrollBy(0, window.innerHeight); // scroll to the bottom of the page
       // wait for 2 seconds to load more elements

      });
      await page.waitForTimeout(2000);
      //capture the current height of the page after scrolling
      const currentheight =await page.evaluate(() => {
        return document.body.scrollHeight; // scroll to the bottom of the page to load more elements
      })
      console.log("Previous height: ",previousheight);
      console.log("Current height: ",currentheight);
      if(currentheight === previousheight)
      {
         break;
      }
      previousheight = currentheight;
   }
console.log("Reached the end of the page");
if(!bookFound){
   console.log("Book not found");

}
});
```

42. [Test] File: keyboardaction.spec.ts

```
//Keyboard Action
//it is used to perform keyboard actions like key press, key down, key up etc.
//keyboard methods:
//keyboard methods are used to perform keyboard actions like key press, key down, key up etc.
//keyboard.press(key, options) - it is used to press a key.
//keyboard.down(key, options) - it is used to press a key down.
//keyboard.up(key, options) - it is used to release a key.
//inserttext,down,press,type,up are the methods of keyboard class which are used to perform keyboard actions.
import { test, expect } from '@playwright/test';
test('Keyboard Action', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/');
    const input1=page.locator('#input1');
//1.focus on the input field
    await input1.focus(); //focus|click
//2.provide some text in the input field
    await page.keyboard.insertText('Hello World'); //type|insertText
//3.ctrl+a to select all the text
    await page.keyboard.down('Control');
    await page.keyboard.press('a');
    await page.keyboard.up('Control');
//4.ctrl+c to copy the text
    await page.keyboard.down('Control');
    await page.keyboard.press('c');
    await page.keyboard.up('Control');
//5.press tab twice to move to the next input field
    await page.keyboard.press('Tab');
    await page.keyboard.press('Tab');
//6.ctrl+v to paste the text in the next input field
    await page.keyboard.down('Control');
    await page.keyboard.press('v');
    await page.keyboard.up('Control');
//7.press tab twice to move to the next input field
    await page.keyboard.press('Tab');
    await page.keyboard.press('Tab');
//8.ctrl+v to paste the text in the next input field
    await page.keyboard.down('Control');
    await page.keyboard.press('v');
    await page.keyboard.up('Control');
await page.waitForTimeout(5000);
});
test.only('Keyboard Action -simple mode', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/');
    const input1=page.locator('#input1');
//1.focus on the input field
    await input1.focus(); //focus|click
```

```
//2.provide some text in the input field
    await page.keyboard.insertText('Hello World'); //type|insertText
//3.ctrl+a to select all the text
    await page.keyboard.press('Control+a'); //shortcut for select all
//4.ctrl+c to copy the text
    await page.keyboard.press('Control+c'); //shortcut for copy
//5.press tab twice to move to the next input field
    await page.keyboard.press('Tab');
    await page.keyboard.press('Tab');
//6.ctrl+v to paste the text in the next input field
    await page.keyboard.press('Control+v'); //shortcut for paste
//7.press tab twice to move to the next input field
    await page.keyboard.press('Tab');
    await page.keyboard.press('Tab');
//8.ctrl+v to paste the text in the next input field
    await page.keyboard.press('Control+v'); //shortcut for paste
await page.waitForTimeout(5000);
});
```

43. [Test] File: locators.spec.ts

```
//Locators are the central piece of Playwright's auto-waiting and retry-ability. In a nutshell, locators represent a way to find element(s) on the page at any moment.
//Locators are lazy: they do not perform any action or query until an action is performed on them. This allows locators to always point to the most up-to-date element matching the selector, even if the DOM changes between the time the locator is created and the time an action is performed on it.
//Locators are auto-waiting: Playwright automatically waits for the element to be ready before performing actions on it. This includes waiting for the element to be attached to the DOM, visible, stable, and enabled.
//Locators are retry-able: If an action fails because the element is not ready, Playwright will retry the action until it succeeds or a timeout is reached.
/*page.getByRole() to locate by explicit and implicit accessibility attributes.
page.getByText() to locate by text content.
page.getByLabel() to locate a form control by associated label's text.
page.getByPlaceholder() to locate an input by placeholder.
page.getByAltText() to locate an element, usually image, by its text alternative.
page.getByTitle() to locate an element by its title attribute.
page.getByTestId() to locate an element based on its data-testid attribute (other attributes can be configured).*/
import {expect, Locator, test} from '@playwright/test';
test("Verify locators on the page", async ({ page }) => {
  await page.goto("https://demo.nopcommerce.com/");
 // page.getByAltText() to locate an element, usually image, by its text alternative.
  const logo: Locator = page.getByAltText("nopCommerce demo store");
  await expect(logo).toBeVisible();
//await logo.click();
//page.getByText() to locate by text content.
//used for static text on the page like headings, paragraphs, links, etc.
//const text:Locator= page.getByText("Welcome to our store");
//await expect(text).toBeVisible();
  await expect(
    page.getByText("Welcome to our store")
  ).toBeVisible();
//page.getByRole() to locate by explicit and implicit accessibility attributes.
//role includes button, link, checkbox, textbox, heading, etc.
  await Promise.all([
    page.waitForURL(/\/register/),
    page.getByRole("link", { name: "Register" }).click(),
  ]);
  // □    Reliable assertion: URL (not blocked by Cloudflare UI)
  await expect(page).toHaveURL(/\/register/);
//page.getByLabel() to locate a form control by associated label's text.
//used for input fields, text areas, dropdowns etc.
 await expect(page.getByLabel("First name:")).toBeVisible();
  await page.getByLabel("First name:").fill("John");
  await page.getByLabel("Last name:").fill("Wick");
  await page.getByLabel("Email:").fill("johnwick@gmail.com");
  //page.getByPlaceholder() to locate an input by placeholder.
```

```
  //best for input fields where placeholder text is descriptive
 await page.getByPlaceholder("Search Store").fill("laptop");
});
```

44. [Test] File: mouseaction.spec.ts

```
//Mouse action tests
// it is used to handle the mouse actions in the browser by using the mouse object
import { test, expect } from '@playwright/test';
test('Mouse action demo', async ({ page }) => {
await page.goto('https://testautomationpractice.blogspot.com/');
const pointme=page.locator('.dropbtn');
await pointme.hover();
const laptops=page.locator('.dropdown-content a:nth-child(2)');
laptops.hover();
await page.waitForTimeout(5000);
});
test('Mouse Right click action demo', async ({ page }) => {
await page.goto('https://swisnl.github.io/jQuery-contextMenu/demo.html');
const button=page.locator('span.context-menu-one');
await button.click({button:'right'}); // right click action
await page.waitForTimeout(3000);
});
test('Double click action demo', async ({ page }) => {
await page.goto('https://testautomationpractice.blogspot.com/');
const btncopy=page.locator('button[ondblclick="myFunction1()"]');
await btncopy.dblclick(); // double click action
const textfield=page.locator('#field2');
await expect(textfield).toHaveValue('Hello World!'); // assertion to verify the double click action
await page.waitForTimeout(3000);
});
test.only('Drag and Drop action demo', async ({ page }) => {
  await page.goto('https://codepen.io/EpsilonDeltaCriterion/pen/jLoPgE');
  // Target ONLY the result iframe
  const frame = page.frameLocator('iframe.result-iframe');
  const rome = frame.locator('#box6');
  const italy = frame.locator('#box106');
  //await rome.dragTo(italy);
   await rome.hover();
   await page.mouse.down();
    await italy.hover();
    await page.mouse.up();
  //approach 2: using mouse move and mouse down and mouse up actions
  const washington = frame.locator('#box3');
  const usa = frame.locator('#box103');
  await washington.dragTo(usa); // using dragTo method
  await page.waitForTimeout(5000);
});
```

45. [Test] File: mytest.spec.ts

```
//syntax
/*test( "title",()=> {
    //step 1
    //step 2
    //assertion
});*/
/*test("Verify the title of the page",async ({page})=> {
    //step 1: navigate to the url
     await page.goto("https://demo.opencart.com/");
     let title:string = await page.title();
     console.log("title of the page is: ", title);
   await  expect(page).toHaveTitle("Your Store");
}); */
//await is used to pause code execution until an asynchronous operation finishes, so the next line runs with valid data and state.
import {expect, test} from '@playwright/test';
test("Verify the title of the page", async ({ page }) => {
  await page.goto(
    "https://gauravkhurana.in/test-automation-play/",
    { waitUntil: "domcontentloaded" }
  );
  await expect(page).toHaveTitle("Test Automation Practice Hub");
  const title = await page.title();
  console.log("Title of the page is:", title);
  const url = page.url();
  console.log("Current URL is:", url);
  await expect(page).toHaveURL(
    "https://gauravkhurana.in/test-automation-play/"
  );
});
```

46. [Test] File: mytest2.spec.ts

```
import { test, expect } from '@playwright/test';
test("Verify the title of the page", async ({ page }) => {
  await page.goto("https://testautomationpractice.blogspot.com/");
  await expect(page).toHaveTitle(/Test Automation Practice Hub/);
  console.log("Title:", await page.title());
  console.log("URL:", page.url());
  await expect(page).toHaveURL(/testautomationpractice\.blogspot\.com/);
});
```

47. [Test] File: paginationtable.spec.ts

```
//paginationtable
//A pagination table is a table that splits large data into multiple pages instead of showing everything at once.
import { test, expect, Locator } from "@playwright/test";
test("paginationtable", async ({ page }) => {
  await page.goto("https://datatables.net/examples/basic_init/zero_configuration.html");
let hasmorepages=true;
//const rows=await page.locator("#example tbody tr").all();
/*for(let row of rows)
{
   console.log(await row.innerText());
}*/
while(hasmorepages)
{
    const rows=await page.locator("#example tbody tr").all();
    for(const row of rows)
    {
      console.log(await row.innerText());
    }
await page.waitForTimeout(2000);
//button[aria-label='Next']
//button[aria-controls='example']:has-text(",")
//button[aria-controls='example']:nth-child(9  )
const nextButton:Locator=page.locator("button[aria-label='Next']");
const isDisabled=await nextButton.getAttribute('class');
if(isDisabled?.includes("disabled"))
{
   hasmorepages=false;
}
else{
   await nextButton.click();
}
}
//await page.waitForTimeout(5000);
});
test("Filter rows and check the rows count", async ({ page }) => {
  await page.goto("https://datatables.net/examples/basic_init/zero_configuration.html");
  const dropdowns:Locator=await page.locator('#dt-length-0');
  await dropdowns.selectOption({label:'25'});
//Approach 1
  const rows=await page.locator("#example tbody tr").all();
  expect(rows.length).toBe(25);//assertion
//Approach 2
  const rows2=await page.locator("#example tbody tr");
  await expect(rows2).toHaveCount(25);
});
```

```
test.only("Searching rows and check the rows count", async ({ page }) => {
  await page.goto("https://datatables.net/examples/basic_init/zero_configuration.html");
const searchbox:Locator=page.locator('#dt-search-0');
await searchbox.fill('Paul Byrd');
await page.waitForTimeout(5000);
const rows=await page.locator("#example tbody tr").all();
if(rows.length>=1)
{
   let matchfound=false;
   for(let row of rows)
   {
      const text=await row.innerText();
      if(text.includes('Paul Byrd'))
      {
         console.log("Record exist- found");
         matchfound=true;
         break;
      }
   }
   //expect(matchfound).toBe(true);
   expect(matchfound).toBeTruthy();
}
else{
   console.log("Record not found");
}
});
```

48. [Test] File: paralleltest.spec.ts

```
//parallel testing
//Parallel Testing means running multiple test cases at the same time on different environments instead of running them one by one.
//This helps reduce the total testing time and increase efficiency.
/* Run tests in files in parallel */
//-->fullyParallel: true, //done by rohith
/* Run tests in files in serial */
//-->fullyParallel: false, //done by rohith
//in serial mode 1 worker should be there
//-->we can change worker number in config file as we need
//-->fullyParallel: true,we can also run this inside specific browser
//paste this code into specifin broswer code section
//run time also we can specify the worker
//--> npx playwright test paralleltest.spec.ts --workers 4
import { test} from '@playwright/test';
//test.describe.configure({mode:'serial'});
//ctest.describe.configure({mode:'parallel'});
test.describe('Group1',async() => {
   test('test1', async () => {
      console.log('This is test 1');
   });

   test('test2', async () => {
      console.log('This is test 2');
   });
   test('test3', async () => {
      console.log('This is test 3');
   });
   test('test4', async () => {
      console.log('This is test 4');
   });
   test('test5', async () => {
      console.log('This is test 5');
});
});
```

49. [Test] File: parameterization.spec.ts

```
//parametrization
//Parameterization in testing means running the same test case multiple times with different input data instead of writing separate test cases for each input.
import { test, expect } from '@playwright/test';
//test data
const searchitem:string[]=['laptop','Gift card','smartphone','monitor'];
//using for-of-loop
/*for(const item of searchitem)
{
//-->'Search Test for ${item}` this is for evry time loop run without this element it will give duplicate error
test(`Search Test for ${item}`, async ({ page }) => {
await page.goto('https://demowebshop.tricentis.com/');
await page.locator('#small-searchterms').fill(item);
await page.locator("input[value='Search']").click();
await expect.soft(page.locator('h2 a').nth(0)).toContainText(item,{ignoreCase:true});
});
}*/
//using forEach function
/*searchitem.forEach((item)=>{
test(`Search Test for ${item}`, async ({ page }) => {
await page.goto('https://demowebshop.tricentis.com/');
await page.locator('#small-searchterms').fill(item);
await page.locator("input[value='Search']").click();
await expect.soft(page.locator('h2 a').nth(0)).toContainText(item,{ignoreCase:true});
});
});*/
//describe block
test.describe("Searching itesms",async()=>{
 searchitem.forEach((item)=>{
test(`Search Test for ${item}`, async ({ page }) => {
await page.goto('https://demowebshop.tricentis.com/');
await page.locator('#small-searchterms').fill(item);
await page.locator("input[value='Search']").click();
await expect.soft(page.locator('h2 a').nth(0)).toContainText(item,{ignoreCase:true});
});
});
});
```

50. [Test] File: parametertest.spec.ts

```
import { test, expect } from '@playwright/test';
const loginTestData: string[][] = [
  ["laura.taylor1234@example.com", "test123", "valid"],
  ["invaliduser@example.com", "test321", "invalid"],
  ["validuser@example.com", "testxyz", "invalid"],
  ["", "", "invalid"],
];
test.describe("Login data driven Test", () => {
  for (const [email, password, validity] of loginTestData) {
    test(`Login test for ${email} and ${password}`, async ({ page }) => {
      await page.goto('https://demowebshop.tricentis.com/login');
      // Fill login form
      await page.locator('#Email').fill(email);
      await page.locator('#Password').fill(password);
      await page.locator('input[value="Log in"]').click();
      if (validity.toLowerCase() === 'valid') {
        const logoutLink = page.locator('a[href="/logout"]');
        await expect(logoutLink).toBeVisible({ timeout: 5000 });
      } else {
        const errorMessage = page.locator('.validation-summary-errors');
        await expect(errorMessage).toBeVisible({ timeout: 5000 });
        await expect(page).toHaveURL('https://demowebshop.tricentis.com/login');
      }
    });
  }
});
```

51. [Test] File: paratestCSV.spec.ts

```
//->npm install csv-parse
import { test, expect } from '@playwright/test';
import fs from 'fs';
import { parse } from 'csv-parse/sync';
//reading data from CSV
const csvPath="json-and-csv-testdata/Data.csv";
const fileContent=fs.readFileSync(csvPath,'utf-8');
const records=parse(fileContent,{
  columns:true,
  skip_empty_lines:true
});
//main test
test.describe("Login data driven Test", async() => {
for (const data of records) {
    test(`Login test with email: ${data.email} and password: ${data.password}`, async ({ page }) => {
      await page.goto('https://demowebshop.tricentis.com/login');
      // Fill login form
      await page.locator('#Email').fill(data.email);
      await page.locator('#Password').fill(data.password);
      await page.locator('input[value="Log in"]').click();
      if (data.validity.toLowerCase() === 'valid') {
        const logoutLink = page.locator('a[href="/logout"]');
        await expect(logoutLink).toBeVisible({ timeout: 5000 });
      } else {
        const errorMessage = page.locator('.validation-summary-errors');
        await expect(errorMessage).toBeVisible({ timeout: 5000 });
        await expect(page).toHaveURL('https://demowebshop.tricentis.com/login');
      }
    });
  }

});
```

52. [Test] File: paratestJSON.spec.ts

```
import { test, expect } from '@playwright/test';
import fs from 'fs';
//reading data from json
const jsonPath="jsontestdata/data.json"
const loginTestData:any=JSON.parse(fs.readFileSync(jsonPath,'utf-8'));
//main test
test.describe("Login data driven Test", async() => {
for (const {email, password, validity} of loginTestData) {
   test(`Login test with email: ${email} and password: ${password}`, async ({ page }) => {
     await page.goto('https://demowebshop.tricentis.com/login');
     // Fill login form
     await page.locator('#Email').fill(email);
     await page.locator('#Password').fill(password);
     await page.locator('input[value="Log in"]').click();
     if (validity.toLowerCase() === 'valid') {
       const logoutLink = page.locator('a[href="/logout"]');
       await expect(logoutLink).toBeVisible({ timeout: 5000 });
     } else {
       const errorMessage = page.locator('.validation-summary-errors');
       await expect(errorMessage).toBeVisible({ timeout: 5000 });
       await expect(page).toHaveURL('https://demowebshop.tricentis.com/login');
     }
   });
 }

});
```

53. [Test] File: paratestXLSX.spec.ts

```
//->npm install xlsx
import { test, expect } from '@playwright/test';
import fs from 'fs';
import * as XLSX from 'xlsx';
//load excel file
const excelPath="json-and-csv-testdata/Data.xlsx";
const workbook=XLSX.readFile(excelPath);
const sheetName=workbook.SheetNames[0];
const worksheet=workbook.Sheets[sheetName];
const records=XLSX.utils.sheet_to_json(worksheet);
//convert sheet into json
const loginTestData:any=XLSX.utils.sheet_to_json(worksheet);
//main test
test.describe("Login data driven Test", async() => {
for (const  {email, password, validity } of loginTestData) {
   test(`Login test with email: "${email}" and password: "${password}"`, async ({ page }) => {
     await page.goto('https://demowebshop.tricentis.com/login');
     // Fill login form
     await page.locator('#Email').fill(email);
     await page.locator('#Password').fill(password);
     await page.locator('input[value="Log in"]').click();
     if (validity.toLowerCase() === 'valid') {
       const logoutLink = page.locator('a[href="/logout"]');
       await expect(logoutLink).toBeVisible({ timeout: 5000 });
     } else {
       const errorMessage = page.locator('.validation-summary-errors');
       await expect(errorMessage).toBeVisible({ timeout: 5000 });
       await expect(page).toHaveURL('https://demowebshop.tricentis.com/login');
     }
   });
 }

});
```

54. [Test] File: pomtest.spec.ts

```
//Page object model
//Page Object Model in TypeScript is a structured approach where web pages are modeled as classes with strongly typed elements and reusable methods, improving code maintainability, readability, and scalability in automation frameworks.
//Page Object Model (POM) in TypeScript is a design pattern used in test automation where each web page of an application is represented as a TypeScript class, containing:
//Locators (elements on the page)
//Methods (actions that can be performed on those elements)
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/loginpage';
import { HomePage } from '../pages/homepage';
import { CartPage } from '../pages/cartpage';
test("User can login,add a product to the cart", async ({ page }) => {
await page.goto("https://www.demoblaze.com/index.html")
//Login Page
const loginPage = new LoginPage(page);
await loginPage.clickLoginLink();
//await page.pause();
await loginPage.enterUserName("pavanol");
await loginPage.enterPassword("test@123");
await loginPage.clickOnLoginButton();
//loginPage.performLogin("pavanol","test@123");
//Homepage
const homePage = new HomePage(page);
await homePage.addProductToCart("Samsung galaxy s6");
await page.waitForTimeout(2000);
await homePage.gotoCart();
await page.waitForTimeout(2000);
//cart page
const cartPage = new CartPage(page);
const isProductInCart = await cartPage.checkProductInCart("Samsung galaxy s6");
expect(isProductInCart).toBe(true);
})
```

55. [Test] File: popups.spec.ts

```
//popups
//popups is a library for creating popups in the browser. It is built on top of the native browser APIs and provides a simple and consistent interface for creating and managing popups.
import { test, expect,Locator,Page,chromium } from '@playwright/test';
test('Popups demo', async ({browser}) => {
   const context=await browser.newContext();
const page=await context.newPage();
await page.goto('https://testautomationpractice.blogspot.com/');
//multiple popups
//page.waitForEvent('popup');
//await page.locator('#PopUp').click();
await Promise.all([page.waitForEvent('popup'), page.locator('#PopUp').click()]);
const allpopupwindows=context.pages();
console.log("Number of popups opened:",allpopupwindows.length);
console.log(allpopupwindows[0].url());
console.log(allpopupwindows[1].url());
for(const pw of allpopupwindows)
   {
const title=await pw.title();
if(title.includes("Playwright"))
{
   await pw.locator('.getStarted_Sjson').click();
   await pw.close();
}
   }
   await page.waitForTimeout(5000);
});
```

56. [Test] File: reporter.spec.ts

```
import { test, expect } from '@playwright/test';
test.beforeEach('Launchinf app', async ({ page }) => {

await page.goto('https://demowebshop.tricentis.com/');
});
test('logotest', async ({ page }) => {
  await expect(page.locator("img[alt='Tricentis Demo Web Shop']")).toBeVisible();
});
test('title test', async ({ page }) => {
  expect(await page.title()).toContain("Demo Web Shop");
});
test('search test', async ({ page }) => {
  await page.locator('#small-searchterms').fill("laptop"); // fill teh text in search box
  await page.locator("input[value='Search']").click(); // click on the button
  await expect.soft(page.locator('h2 a').nth(0)).toContainText("laptop", { ignoreCase: true });
});
//1.using config file
//-->reporter:[['html',{open:'always','outputFolder':'html-report'}]]
//2.npx playwright test reporter.spec.ts --reporter=html
//list reporter
//3.reporter:[['html',{open:'always','outputFolder':'html-report'}],['list']]
//line reporter
//4.eporter:[['html',{open:'always','outputFolder':'html-report'}],['list'],['line']]
//dot reporter
//reporter:[['html',{open:'always','outputFolder':'html-report'}],['list'],['line'],['dot']]
//junit reporter
//reporter:[['html',{open:'always','outputFolder':'html-report'}],['list'],['line'],['dot'],['junit',
{outputFile:'junit.xml'}]]
//json report
//reporter:[['html',{open:'always','outputFolder':'html-report'}],['list'],['line'],['dot'],['junit',
{outputFile:'junit.xml'}],['json',{outputFile:'result.json'}]]
```

57. [Test] File: screenshot.spec.ts

```
//screenshot capture
//to capture screenshot, use page.screenshot() method
//it captures the screenshot of the current page and saves it in the specified path
//it returns a buffer of the screenshot which can be used to save the screenshot in the desired location
import { test, expect } from '@playwright/test';
test('Screenshot Capture', async ({ page }) => {
    await page.goto('https://demowebshop.tricentis.com/');
    const timestamp=Date.now(); //get the current timestamp
    //capture screenshot of the entire page
    await page.screenshot({path:'screenshot/'+'homepage_'+timestamp+'.png'}); //provide the path where the
screenshot will be saved and set fullPage to true to capture the entire page
//full page screenshot
await page.screenshot({path:'screenshot/'+'homepage_fullpage_'+timestamp+'.png',fullPage:true}); //provide
the path where the screenshot will be saved and set fullPage to true to capture the entire page

//to capture screenshot of a specific element, use locator.screenshot() method
const logo= page.locator("img[alt='Tricentis Demo Web Shop']");
   await logo.screenshot({path:'screenshot/'+'logo'+timestamp+'.png'}); //provide the path where the screenshot
will be saved
//to capture perticular section of the page, use page.screenshot() method with clip option
await page.screenshot({path:'screenshot/'+'featuredproducts_'+timestamp+'.png', clip:
{x:0,y:0,width:800,height:400}}); //provide the path where the screenshot will be saved and set clip option to
capture the specific section of the page
    console.log("Screenshot captured successfully...."); //print the status message
});
//to capture screenshot globally for all tests, we can set the screenshot option in the playwright.config.ts file
test.only('screenshot from the config file', async ({ page }) => {
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginpassword').fill('test@13');
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.getByRole('link', { name: 'Log out' })).toBeVisible();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');


});
```

58. [Test] File: shadowdom.spec.ts

```
//Shadow Dom Test
//This test verifies that the application correctly interacts with elements inside a shadow DOM.
//Shadow DOM is a web standard that allows developers to encapsulate their HTML, CSS, and JavaScript, creating a separate DOM tree for a component. This helps in avoiding style and behavior conflicts with the main document.
import { test, expect } from '@playwright/test';
test('Shadow Dom Test', async ({ page }) => {
   await page.goto('https://books-pwakit.appspot.com/'); //navigate to the application under test

   await page.locator('#input').fill('Playwright automation'); //fill the search input field with the search term
   await page.keyboard.press('Enter'); //press enter to perform the search
   await page.waitForTimeout(3000); //wait for the search results to load
   const booksfound=await page.locator('h2.title').all();
   console.log("Books found: ",booksfound.length); //print the number of books found
   //assert that at least one book is found
   expect(booksfound.length).toBe(5); //assert that at least one book is found
   console.log("Books found successfully...."); //print the status message
});
test.only('shadow dom 2nd test', async ({ page }) => {
   await page.goto("https://shop.polymer-project.org/"); //navigate to the application under test
   await page.locator("a[aria-label=\"Men's Outerwear Shop Now\"]").click(); //click on the men's outerwear category link

   await page.locator('div.title').first().waitFor(); //wait for the product titles to be visible
   const productfound=await page.locator('div.title').all(); //get all the product titles
   console.log("Products found: ",productfound.length); //print the number of products found
   //assert that at least one product is found
   expect(productfound.length).toBe(16); //assert that at least one
   await page.waitForTimeout(5000); //wait for some time before closing the browser
   console.log("Products found successfully...."); //print the status message
});
```

59. [Test] File: sorted.spec.ts

```
//sorted dropdown
import{test,expect,Locator} from '@playwright/test';
test("Single select drop down",async({page})=>{
await page.goto('https://testautomationpractice.blogspot.com/');
//const Dropdownoption:Locator=page.locator('#colors>option');
const Dropdownoption:Locator=page.locator('#animals>option');//try animals
console.log(await Dropdownoption.allTextContents());
const  optiontext:string[]=(await Dropdownoption.allTextContents()).map(text=>text.trim());
const originallist:string[]=[...optiontext];//... spread operator
const sortedlist:string[]=[...optiontext.sort()]; //array sort is mutable
console.log("original list",originallist);
console.log("sorted list",sortedlist);
expect(originallist).toEqual(sortedlist); //this will give error
 //await page.waitForTimeout(3000);
});
```

60. [Test] File: statictable.spec.ts

```
import{expect,Locator,test} from "@playwright/test";
test("Static Table",async({page})=>{
await page.goto("https://testautomationpractice.blogspot.com/");
const table:Locator=page.locator("table[name='BookTable'] tbody");
await expect(table).toBeVisible();
//1)count number of rows in a table:
//const rows:Locator=page.locator("table[name='BookTable'] tbody tr");// returns al the rows including heade
const rows:Locator=table.locator("tr");
await expect(rows).toHaveCount(7);  //approach 1
const rowcounts:number=await rows.count();
console.log("Number of rows in a table",rowcounts);
expect(rowcounts).toBe(7);//approach 2
//2)Count number of headers/column
//const columns:Locator=page.locator("table[name='BookTable'] tbody tr th");
const columns:Locator=rows.locator("th");
await expect(columns).toHaveCount(4);
const columncounts:number=await columns.count();
console.log("Number of columns in a table",columncounts);
expect(columncounts).toBe(4);
//3)Read all data from 2nd row(index 2 means 3rd row including header)
const secondrowcells:Locator=rows.nth(2).locator("td");
const secondrowText:string[]=await secondrowcells.allInnerTexts();
console.log("2nd row data:",secondrowText); //2nd row data: [ 'Learn Java', 'Mukesh', 'Java', '500' ]
await expect(secondrowcells).toHaveText([ 'Learn Java', 'Mukesh', 'Java', '500' ]);//assertion
console.log("Printing 2nd row data.....");
for(let text of secondrowText)
   {
      console.log(text);
   }
//4)Read all data from the table(Excluding header)
console.log("Printing all data from the table.....");
const allrowdata=await rows.all();
console.log("BookName\tAuthor\tSubject\tPrice");
for(let row of allrowdata.slice(1)) //slice skip headwr row
{
   const cols=await row.locator('td').allInnerTexts();
   console.log(cols.join('\t'));
}
//5)print book names where author is Mukesh
console.log("Printing book names where author is Mukesh.....");
const mukeshbook:string[]=[];
for(let row of allrowdata.slice(1))
{
   const cells=await row.locator('td').allInnerTexts();
   const author=cells[1];
```

```
    const BookName=cells[0];
    if(author==='Mukesh')
    {
        console.log(`${author}\t${BookName}`);
        mukeshbook.push(BookName);

    }
}
expect(mukeshbook).toHaveLength(2);//ASSERTION
//Calculate total price of all book
let totalprice:number=0;
console.log("Printing total price of all book.....");
for(let row of allrowdata.slice(1))
{
    const cells=await row.locator('td').allInnerTexts();
    const price=cells[3];
    totalprice=totalprice+parseInt(price);
}
console.log("Total price:",totalprice);
expect(totalprice).toBe(7100);//assertion
});
```

61. [Test] File: tabs.spec.ts

```
//tabs
//tabs should be able to switch between different content when clicked
//tabs is a component that allows users to switch between different content by clicking on the tab headers
import{test,expect,chromium,Locator} from '@playwright/test';
test('Tabs demo', async () => {
const browser=await chromium.launch();
const context=await browser.newContext();
//creating 1 page
const parentpage=await context.newPage();
parentpage.goto('https://testautomationpractice.blogspot.com/');
// 2 statements should be executed at the same time, one is waiting for the new page to open and the other is clicking on the button that opens the new page
//context.waitForEvent('page'); //pending,fullfilled,rejected
//parentpage.locator("button:has-text('New tab')").click(); //opens new tab6
const [childpage]=await Promise.all([context.waitForEvent('page'),parentpage.locator("button:has-text('New tab')").click()]);
//approach 1:switch between pages and get the title of the page
const pages=context.pages();
console.log("Number of pages created:",pages.length);
console.log("Title of the parent page:",await pages[0].title());
console.log("Title of the child page:",await pages[1].title());
//approach 2:alternative way to switch between pages and get the title of the page
console.log("Title of the parent page:",await parentpage.title());
console.log("Title of the child page:",await childpage.title());
await parentpage.waitForTimeout(5000);
await childpage.waitForTimeout(5000);
});
```

62. [Test] File: tagging.spec.ts

```
//Tagging
//Tagging is the process of assigning labels (tags) to test cases so that specific groups of tests can be easily identified, organized, and executed during test runs.
//Tags help testers run only selected tests based on categories like smoke, regression, or sanity.
//to run all sanity check to specific test
//to run--> npx playwright test tagging.spec.ts --grep "@sanity"
//npx playwright test tagging.spec.ts --grep "@regression"
//to run both -->should have both sanity and regression
//-->npx playwright test tagging.spec.ts --grep(?=.*@sanity)(?=.*@regression)
//to run either sanity or regression
//-->npx playwright test tagging.spec.ts --grep "@sanity|@regression"
//run sanity test which are not belong to regression
//npx playwright test tagging.spec.ts --grep "@sanity" --grep invert "@regression"
//run using config file
//grep:/@sanity/
import { test, expect } from '@playwright/test';
//1.@sanity -->sanity is user defined
test('@sanity @regression Check the title of the home page', async ({ page }) => {
  await page.goto('https://www.google.com/');
  await expect(page).toHaveTitle('Google');
    console.log('test 1 is running...');
});
//2.{tag:'@sanity'}
test('Check the title of the home page',{tag:'@sanity'} ,async ({ page }) => {
  await page.goto('https://www.google.com/');
  await expect(page).toHaveTitle('Google');
console.log('test 2 is running...');
});
//3.{tag:'@regression'}
test('Check the navigation to the store page',{tag:'@regression'} ,async ({ page }) => {
  await page.goto('https://store.google.com/?hl=en-IN&pli=1');
  await expect(page.getByText('Popular on the Google Store')).toBeVisible();
  console.log('test 3 is running...');
});
//4.{tag:['@sanity','@regression']}
test('Check the top recommendations',{tag:['@sanity','@regression']} ,async ({ page }) => {
  await page.goto('https://store.google.com/?hl=en-IN&pli=1');
  await expect(
    page.getByRole('heading', { name: /Switch to Google Pixel/i })
  ).toBeVisible();
  console.log('test 4 is running...');
});
```

63. [Test] File: tracing.spec.ts

```
//tracing
//tracing is a tool that allows you to trace the execution of your code and see how it is executed. It is useful for debugging and understanding how your code works.
//ways to enable tracing in playwright
//1.using playwright.config.ts file
//2.npx playwright test tracing.spec.ts --headed --trace on
//3. using start and stop tracing in the test file
//to view the trace file
//1.from html report click on the trace file link trcce.zip
//2.npx playwright show-trace trace.zip
//3.utility-->https://trace.playwright.dev/ to view the trace file in the browser(drag and drop the trace.zip file in the browser)
import { test, expect } from '@playwright/test';
test('Tracing', async ({ page,context }) => {
    //start tracing
   context.tracing.start({ screenshots: true, snapshots: true });
  await page.goto('https://demoblaze.com/index.html');
  await page.getByRole('link', { name: 'Log in' }).click();
  await page.locator('#loginusername').fill('pavanol');
  await page.locator('#loginpassword').fill('test@123');
  await page.getByRole('button', { name: 'Log in' }).click();
  await expect(page.getByRole('link', { name: 'Log out' })).toBeVisible();
  await expect(page.locator('#nameofuser')).toContainText('Welcome pavanol');
//stop tracing and export it into a zip file
//run the test
//save the zip file in the root directory of the project
//npx playwright show-trace trace.zip run this to view the trace file in the browser
  await context.tracing.stop({ path: 'trace.zip' });
});
//to open the trace file
//npx playwright show-report
```

64. [Test] File: uploadfile.spec.ts

```
//upload files
//used to upload files to the application under test
//filechooser is a class which is used to handle file upload dialog box
//setInputFiles is a method of filechooser class which is used to set the file to be uploaded
//since file upload dialog box is a system level dialog box which cannot be handled by playwright, we can use setInputFiles method to set the file to be uploaded
import { test, expect } from '@playwright/test';
test('File Upload', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/');
    await page.locator("#singleFileInput").setInputFiles('upload/Day26-Tables-Lab+Assignment.pdf'); //provide the path of the file to be uploaded
   await page.locator("button:has-text('Upload single file ')").click(); //click on the upload button
const msg=await page.locator('#singleFileStatus').textContent(); //get the text of the status message
expect(msg).toContain('Day26-Tables-Lab+Assignment.pdf'); //assert the status message
console.log("Upload successful...."); //print the status message
await page.waitForTimeout(5000);
});
//multiple file upload
test.only('Multiple File Upload', async ({ page }) => {
    await page.goto('https://testautomationpractice.blogspot.com/');
    await page.locator("#multipleFilesInput").setInputFiles(['upload/Day26-Tables-Lab+Assignment.pdf','upload/Day27-DatePickers-Labs.pdf']); //provide the path of the files to be uploaded
    await page.locator("button:has-text('Upload multiple files')").click(); //click on the upload button
    const msg=await page.locator('#multipleFilesStatus').textContent(); //get the text of the status message
    expect(msg).toContain('Day26-Tables-Lab+Assignment.pdf'); //assert the status message
    expect(msg).toContain('Day27-DatePickers-Labs.pdf'); //assert the status message
    console.log("Multiple Upload successful...."); //print the status message
await page.waitForTimeout(5000);
});
```

65. [Test] File: xpath.spec.ts

```
//xpath is syntax used to navigate through elements and attributes in an XML document.
//xpath is used to locate elements in a web page for automation testing
//xpath is used in selenium to locate elements
//xpath is used in playwright to locate elements
//xpath is used in many other automation tools to locate elements
//types of xpath
//1) absolute xpath
//2) relative xpath
//absolute xpath starts from the root element and follows the full path to the target element
//single slash (/) is used in absolute xpath
//like html/body/div/div/p
//relative xpath starts from anywhere in the document and uses double slash (//)
//double slash (//) is used in relative xpath
//like //tagname[@attribute='value']
//like //tagname[text()='textvalue']
//And operator in xpath
//used to combine multiple conditions in a single xpath expression
//syntax: //tagname[@attribute1='value1' and @attribute2='value2']
//example:
//input[@type='submit' and @value='Search']
//Or operator in xpath
//used to specify multiple conditions where at least one must be true
//syntax: //tagname[@attribute1='value1' or @attribute2='value2']
//example:
//input[@type='submit' or @value='Search']
import { expect, test,Locator} from '@playwright/test';
import { count } from 'node:console';
test("XPathe demo in Playwright",async({page})=>{
    await page.goto("https://demowebshop.tricentis.com/");
//absolute xpath
//use xpath or double slash(//) because it take single slash as css xpath= or //html
const absolutelogo:Locator=page.locator("xpath=/html[1]/body[1]/div[4]/div[1]/div[1]/div[1]/a[1]/img[1]");
await expect(absolutelogo).toBeVisible();
//relative xpath
const relativelogo:Locator=page.locator("//img[@alt='Tricentis Demo Web Shop']");
await expect(relativelogo).toBeVisible();
//contains() function in xpath
//used to find elements that contain a specific text or attribute value
//syntax: //tagname[contains(@attribute,'value')]
const products :Locator=page.locator("//h2//a[contains(@href,'computer')]");
await products.first().isVisible();
const productCount:number=await products.count();
console.log("Number of products found with 'computer' in href: "+productCount);
expect(productCount).toBeGreaterThan(0);
//textContent used to get the text inside the element
```

```
//get text from ALL matching elements
const productNames = await products.allTextContents();
console.log(productNames);
//get text from specific element
//by using first(), last(), nth() methods
console.log("First computer related product: ",await products.first().textContent());
console.log("Last computer related product: ",await products.last().textContent());
console.log("Nth computer related product: ",await products.nth(1).textContent());
//by using array index
console.log("first computer related product: ",await products.nth(0).textContent());
console.log("second computer related product: ",await products.nth(1).textContent());
//starts-with() function in xpath
//used to find elements whose attribute value starts with a specific string
//syntax: //tagname[starts-with(@attribute,'value')]
const searchBox:Locator=page.locator("//h2/a[starts-with(@href,'/build')]");
const count: number = await searchBox.count(); //    CALL + await
expect(count).toBeGreaterThan(0);
//text() function in xpath
//used to find elements based on their exact text content
//syntax: //tagname[text()='textvalue']
const reglink:Locator=page.locator("//a[text()='Register']");
await expect(reglink).toBeVisible();
//last() function in xpath
//used to select the last element in a set of matching elements
//syntax: (//tagname)[last()]
const lastProduct:Locator=page.locator("//div[@class='column follow-us']//li[last()]");
console.log("Last product on the page: ",await lastProduct.textContent());
await expect(lastProduct).toBeVisible();
console.log("Text content of last product: ",await lastProduct.textContent());
//position() function in xpath
//used to select an element based on its position in a set of matching elements
//syntax: (//tagname)[position()=n]
const positionitem:Locator=page.locator("//div[@class='column follow-us']//li[position()=3]");
console.log("Third product on the page: ",await positionitem.textContent());
await expect(positionitem).toBeVisible();
});
```

66. [Test] File: xpathaxes.spec.ts

```
//xpath axes
//defines the relationship between the current node and nodes in the document.
//allow you to navigate through the XML or HTML document structure in a flexible way.
//parent
//parent axis selects the parent node of the current node.
//example: /bookstore/book/parent::bookstore
//child
//child axis selects all child nodes of the current node.
//example: /bookstore/child::book
//ancestor
//ancestor axis selects all ancestor nodes (parent, grandparent, etc.) of the current node.
//example: /bookstore/book/ancestor::library
//descendant
//descendant axis selects all descendant nodes (children, grandchildren, etc.) of the current node.
//example: /bookstore/descendant::book
//following
//following axis selects all nodes that come after the current node in the document, excluding descendants.
//example: /bookstore/book/following::author
//following-sibling
//following-sibling axis selects all sibling nodes that come after the current node.
//example: /bookstore/book/following-sibling::book
//preceding
//preceding axis selects all nodes that come before the current node in the document, excluding ancestors.
//example: /bookstore/book/preceding::magazine
//preceding-sibling
//preceding-sibling axis selects all sibling nodes that come before the current node.
//example: /bookstore/book/preceding-sibling::book
//self
//self axis selects the current node itself.
//example: /bookstore/book/self::book
import {test,expect,Locator} from '@playwright/test';
test("xpath axes demo",async ({page})=>{
   await page.goto("https://www.w3schools.com/html/html_tables.asp");
//self axis
const germanycell:Locator=page.locator("//td[text()='Germany']/self::td");
await expect(germanycell).toHaveText("Germany");
//parent axis
const parentRow:Locator=page.locator("//td[text()='Germany']/parent::tr");
await expect(parentRow).toContainText("Maria Anders");
console.log(await parentRow.textContent());
//child axis
const secondRowChildren:Locator=page.locator("//table[@id='customers']/child::tbody/child::tr[2]/child::td");
await expect(secondRowChildren).toHaveCount(3);
//ancestor axis
const ancestorTable:Locator=page.locator("//td[text()='Germany']/ancestor::table");
```

```
await expect(ancestorTable).toHaveAttribute("id","customers");
//descendant axis
const tableDescendants:Locator=page.locator("//table[@id='customers']/descendant::td");
await expect(tableDescendants).toHaveCount(18); //try 15
//following axis
const followingNodes:Locator=page.locator("(//td[normalize-space()='Germany'])[1]/following::td");
await expect(followingNodes).toHaveText("Centro comercial Moctezuma"); //try 12
//following-sibling axis
const followingSiblings:Locator=page.locator("//td[text()='Germany']/following-sibling::td");
await expect(followingSiblings).toHaveText("UK");
//preceding axis
const precedingNodes:Locator=page.locator("(//td[normalize-space()='Germany'])[1]/preceding::td");
await expect(precedingNodes).toHaveText("Maria Anders"); //try 6
});
```